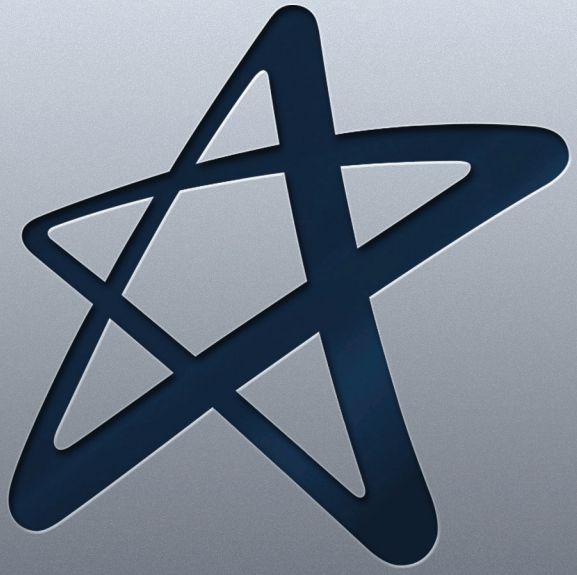
Programação para Dispositivos Móveis



Cruzeiro do Sul Virtual
Educação a Distância

Material Teórico

Introdução ao desenvolvimento para Android
Ferramentas para desenvolvimento Android SDK
Pacotes e classes básicas para desenvolvimento e Layouts

Responsável pelo Conteúdo:
Prof. Ms. Luis Carlos Reis

Revisão Textual:
Prof. Ms. Douglas Almendro

Revisão Textual:
Profa. Ms. Luciene Oliveira da Costa Santos

UNIDADE

Introdução ao desenvolvimento para Android
Ferramentas para desenvolvimento Android SDK
Pacotes e classes básicas para desenvolvimento e Layouts



- Introdução
- Arquitetura do Android
- Google Play
- Play Store
- A evolução da plataforma Android
- Criando um projeto no Android Studio.
- Entendendo a classe R.java
- As tags dos arquivos .xml do Android
- Referências



OBJETIVO DE APRENDIZADO

- Introduzir o desenvolvimento para Android;
- Explicar e demostrar as formas de se criar um app mobile;



ORIENTAÇÕES

Caro(a) aluno(a),

Estamos começando nossos estudos sobre desenvolvimento mobile, mais especificamente, estamos nos referindo sobre Android.

A proposta desta unidade é instalar e configurar o Android Studio e o estudo de criação de um programa para dispositivos móveis utilizando os conceitos iniciais de layout e algumas classes básicas para realizar as tarefas propostas nesta unidade.

Não deixe de assistir, também, à apresentação narrada do conteúdo e de alguns exercícios resolvidos.

Finalmente, e o mais importante, fique atento(a) às atividades avaliativas propostas e ao prazo de realização e envio.

Bom estudo!

Contextualização

Por que desenvolver para Android?

Primeiramente, porque ele é totalmente de graça, ou seja, você não precisa comprar ferramentas, APIs ou quaisquer tipos de software para desenvolver para Android, nem uma plataforma específica ou nem mesmo um celular ou gadget com Android. Só baixar SDK, uma IDE e sair desenvolvendo.

Basicamente, TUDO pode ser feito no Android! Podemos utilizar todos os recursos disponíveis nos equipamentos de celular ou gadget que você possui.

Precisa de mais informações? É só olhar no Android Market e ver a variedade de Apps que existem por aí e imagine que seus Apps podem estar lá também e ainda com possibilidade de ganhos!

Enfim…

Estão curiosos? Vamos colocar a mão na massa e ver que desenvolver para mobile não é um bicho de sete cabeças.

Bom estudo!

Introdução

O poder de processamento dos celulares e outros aparelhos móveis, hoje em dia, são bem avançados e com esse avanço também há sensíveis melhorias nas telas e na usabilidade desses dispositivos.

Por essas razões, plataformas móveis como Blackberry, iOS e Android estão cada vez mais em evidência e o mercado em ascensão. A explosão do Android é a mais recente dentre essas plataformas, e seu crescimento cada vez maior.

Pensando assim, utilizaremos o sistema operacional Android para desenvolvimento em nossas aulas pelo fato de o mesmo ser plataforma livre. Você não precisa comprar ferramentas, APIs ou qualquer tipo de software para desenvolver para Android, nem uma plataforma específica ou nem mesmo um celular ou gadget com Android. Só realizar o download do SDK e uma IDE e pronto.



Figura 1

O Android é um sistema operacional que é executado sobre o núcleo Linux, onde foi inicialmente desenvolvido pela Android Inc., e depois passou para o Google onde foi comprado em 2005 e posteriormente pela Open Handset Alliance.

O Android é a primeira plataforma para aplicações móveis completamente livres e de código aberto (open source), sendo uma plataforma de desenvolvimento para telefones celulares e outros dispositivos móveis.

No Android, cada aplicação é executada em um único processo e cada processo possui uma thread dedicada.

Esse sistema operacional é baseado no núcleo (kernel) 2.6 do Linux, e é responsável por gerenciar a memória, os processos threads e a segurança dos arquivos e pastas, além de redes e drivers.

A Open Handset Alliance (OHA), um grupo de empresas líderes de telefonia móvel, liderado pela Google, está ligado a este projeto. Motorola, HTC, Sony Ericsson, LG, Samsung, Sprint Nextel, Toshiba etc. são algumas das empresas participantes.



Maiores informações sobre a OHA pode ser obtidas no site:
https://goo.gl/tfSYI

A linguagem mais usada para criar aplicativos para o robozinho é o Java e adotaremos a mesma em nossas aulas, onde poderemos controlar o dispositivo via bibliotecas desenvolvidas pela Google.

A linguagem Java é utilizada para construir as aplicações para o Android. Mas não existe uma máquina virtual Java (JVM). Na realidade, o que existe é uma máquina virtual chamada Dalvik que é otimizada para execução em disponíveis móveis.

Depois que o bytecode (class) é compilado, ele é convertido para o formato .dex (Dalvik Executable) que representa a aplicação do Android compilada. Depois os arquivos **.dex** e outros recursos como imagens são compactados em um único arquivo com a extensão **.apk** (Android Package File) que representa a aplicação final, pronta para ser distribuída e instalada.

Diagrama que representa a arquitetura do Android (extraído de https://bit.ly/4dncB9y).

O Android está disponível como código aberto desde 21 de Outubro de 2008. O Google publicou todo o código sob a licença ASF (Apache Software Foundation) que permite que alterações sejam efetuadas no código-fonte para criar produtos customizados sem precisar compartilhar as alterações com ninguém. No entanto, ele depende de uma autorização do próprio Google para poder acessar a biblioteca de aplicativos, Play Store.

O Android Studio possui um plugin chamado ADT (Android Development Tools) para facilitar o desenvolvimento, os testes e a compilação do projeto.

Arquitetura do Android

A arquitetura do Android é composta por várias camadas e componentes, como é possível visualizar na figura abaixo:

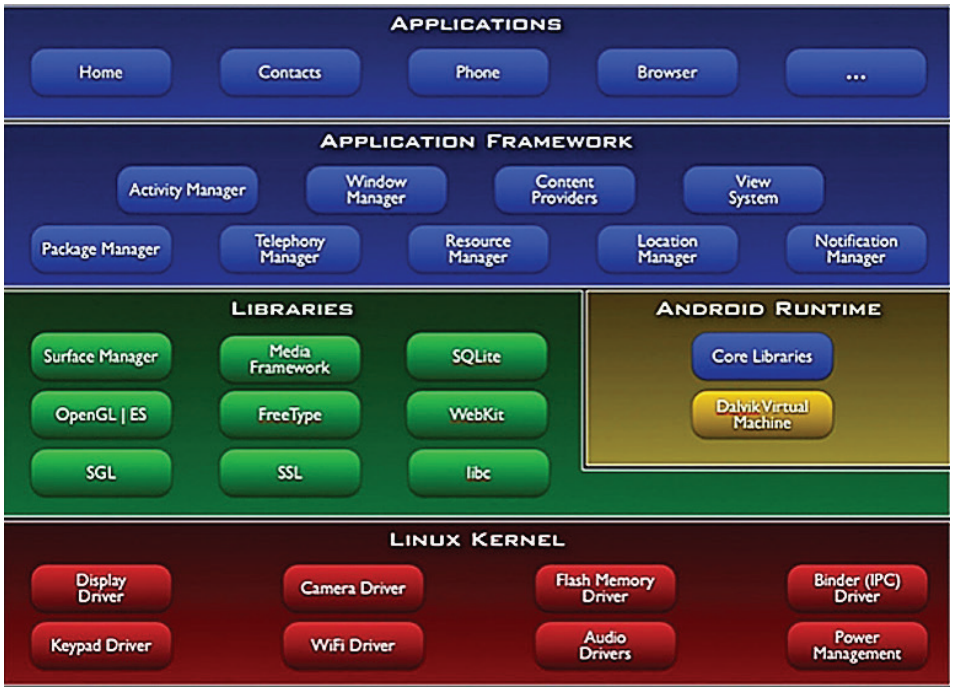


Figura 2

Na camada **Linux Kernel**, está localizado o sistema operacional da plataforma, que é baseado no Linux. Ela é responsável por serviços de mais baixo nível da plataforma, como gerenciamento de memória e processos, segurança etc.

Na camada **Libraries**, estão as bibliotecas nativas escritas em C/C++ que fazem parte da plataforma. Estão nesta camada APIs como o OpenGL ES (para renderização 3D), SQLite (gerenciador de bancos de dados) e suporte a diversos formatos de áudio e vídeo.

A camada **Android Runtime** dá condições para que as aplicações baseadas na plataforma sejam executadas. Um dos componentes desta camada são as core libraries, que disponibilizam uma API Java utilizada para programação (grande parte das funcionalidades encontradas no Java SE estão disponíveis para o Android). Já o outro componente é a Dalvik Virtual Machine, que é uma máquina virtual para suporte à execução de aplicações.

Na camada **Application Framework** ficam as APIs do Android que são normalmente utilizadas pelas aplicações que executam sobre a plataforma. Os gerenciadores de serviços de telefonia, localização e notificação são alguns exemplos do que este framework disponibiliza.

Por fim, a camada **Applications** representa as aplicações que executam sobre a plataforma. Elas podem ser tanto aplicações nativas – como o gerenciador de contatos, navegador, calendário etc. – como aplicações criadas por terceiros. Aliás, para o Android não existe distinção entre aplicações nativas e aplicações desenvolvidas por outras pessoas, e é esta característica que garante a ela o alto grau de flexibilidade e extensibilidade da plataforma.

Google Play

Os aplicativos desenvolvidos para Android podem ser disponibilizados por meio de um servidor pessoal, sendo o mais comum e profissional realizado pelo Google Play.

Google Play é a loja online da Google para distribuição de aplicações, jogos, filmes, música e livros. Anteriormente a loja chamava-se Android Market.

Essa mudança de nome para Google Play ocorreu para que o Google pudesse unificar o Android Market com a loja de músicas, filmes e livros e assim distribuir todo seu conteúdo digital em um só lugar.

As aplicações do Google Play estão disponíveis de graça ou a um custo, para o sistema operacional Android, e podem ser baixados diretamente para um dispositivo móvel ou em um computador pessoal através de seu site.

Essas aplicações são geralmente direcionadas para usuários com base em um atributo particular de hardware de seu dispositivo, como um sensor de movimento (para o movimento dependente de jogos) ou uma câmera frontal (para chamadas

de vídeo online).



Figura 3

Play Store

A aplicação para o sistema Android que permite acesso ao conteúdo chama-se Play Store.

Os aplicativos da Google Play estão disponíveis em diversos países, podendo ser baixados por meio do site da Google Play ou do aplicativo para Android.

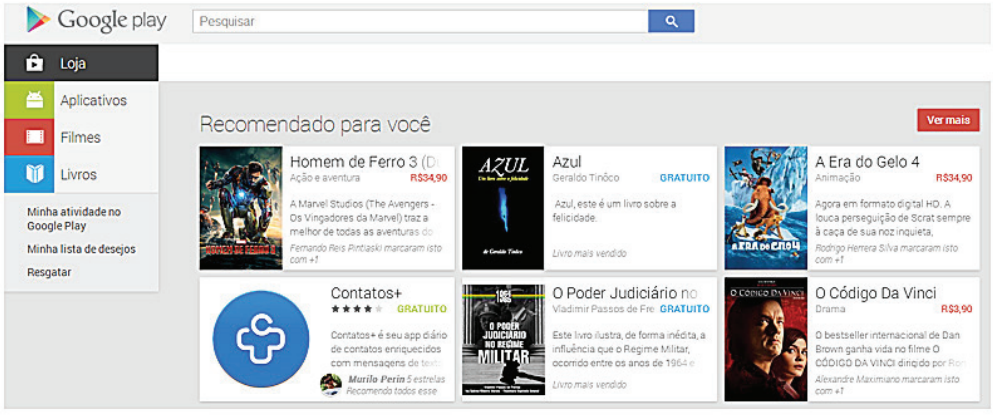


Figura 4

A Evolução da Plataforma Android



Figura 5

Cada plataforma tem um código identificador (API Level) e alguns têm nome de código.

- API Level 1 = Android 1.0
- API Level 2 = Android 1.1
- API Level 3 = Android 1.5 (Cupcake)
- API Level 4 = Android 1.6 (Donut)
- API Level 5 = Android 2.0
- API Level 6 = Android 2.01
- API Level 7 = Android 2.1 (Eclair)
- API Level 8 = Android 2.2 (Froyo)
- API Level 9 = Android 2.3 (Gingerbread)
- API Level 10 = Android 2.3.3
- API Level 11 = Android 3.0 (Honeycomb)
- API Level 12 = Android 3.1 (Honeycomb)
- API Level 13 = Android 3.2 (Honeycomb)
- API Level 14 = Android 4.0 (Ice Cream Sandwich)
- API Level 15 = Android 4.03
- API Level 16 = Android 4.1 (Jelly Bean)
- API Level 17 = Android 4.2 (Jelly Bean)
- API Level 18 = Android 4.3 (Jelly Bean)
- API Level 19 = Android 4.4 (KitKat)
- API Level 20 = Android 4.4W
- API Level 21 = Android 5.0 (LOLLIPOP)

Atualmente na versão do Android é a 5.1:

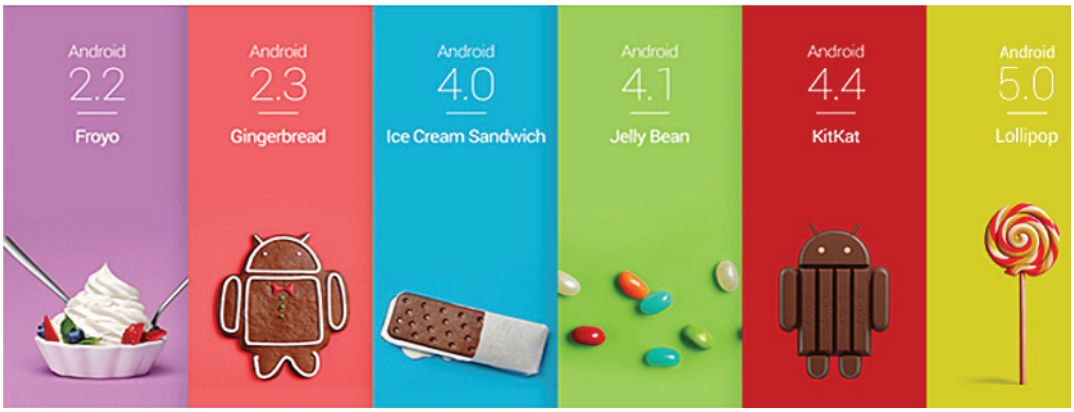


Figura 6

Versão da plataforma Android distribuída por API:

Version	Codename	API	Distribution
2.2	Froyo	8	0.3%
2.3.3 - 2.3.7	Gingerbread	10	5.7%
4.0.3 - 4.0.4	Ice Cream Sandwich	15	5.3%
4.1.x	Jelly Bean	16	15.6%
4.2.x		17	18.1%
4.3		18	5.5%
4.4	KitKat	19	39.8%
5.0	Lollipop	21	9.0%
5.1		22	0.7%

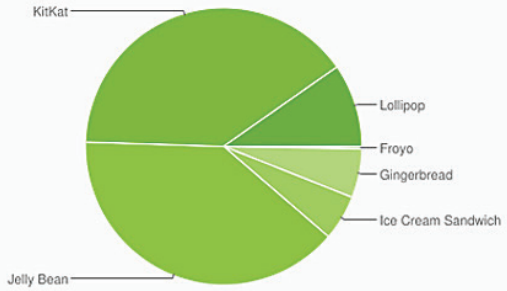


Figura 7

Dados: Maio de 2015. Fonte: https://goo.gl/7ygJx

Onde encontrar uma referência técnica completa do Android?

Documentação dos pacotes de classes das APIs do Android
https://goo.gl/mJ9QHQ

Documentação das classes das APIs do Android
https://goo.gl/BkN46K

Download das versões do Android SDK, plugins, drivers
https://goo.gl/Haaovn

Suporte e instalação do software para Android

Para desenvolvimento com Android, utilizaremos a ferramenta gratuita Android Studio e, atualmente, faz parte do kit de desenvolvimento de software recomendado para desenvolvedores Android.

Para instalar a ferramenta juntamente com o Android, siga os seguintes procedimentos:

- Acessar o site https://goo.gl/Haaovn e realizar o download do pacote do Android Studio.

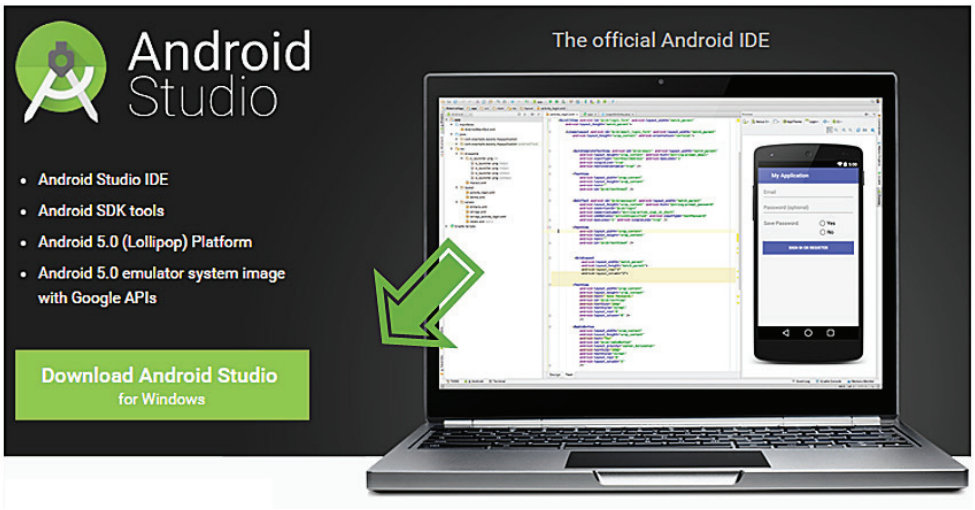


Figura 8

Antes de configurar Android Studio, certifique-se de ter instalado o JDK 6 ou superior (somente o JRE não é suficiente).
JDK 7 é necessário para desenvolver na versão do Android 5.0 e superior.
O mesmo poderá ser obtido no endereço: (Verifique se seu sistema operacional é de 32 ou 64 bits).
http://goo.gl/IxBIV5

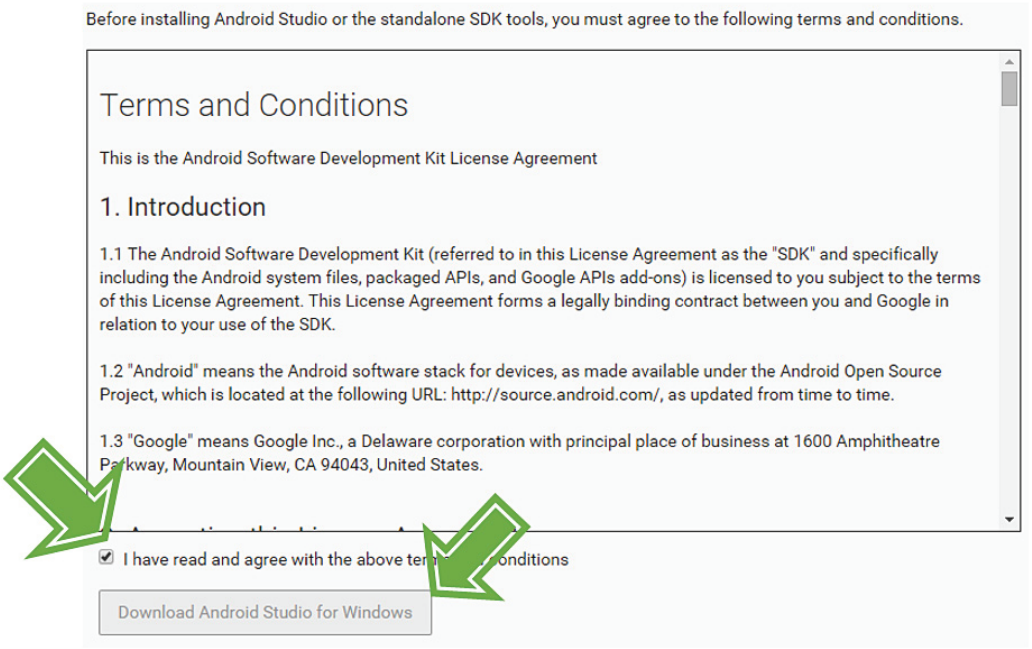


Figura 9

- Como sugestão, crie uma pasta no C: com o nome Android e salve o arquivo para essa pasta.
- O arquivo do download terá o nome android-stuido-bundle-xxx.xxxxxxx-windows.exe



Figura 10

- Neste arquivo, já existe o módulo para desenvolvimento (Android Studio) e o Android SDK, porém vem somente a última API do Android instalada.
- Após instalação execute o Android Studio, onde iremos instalar os demais pacotes do Android.

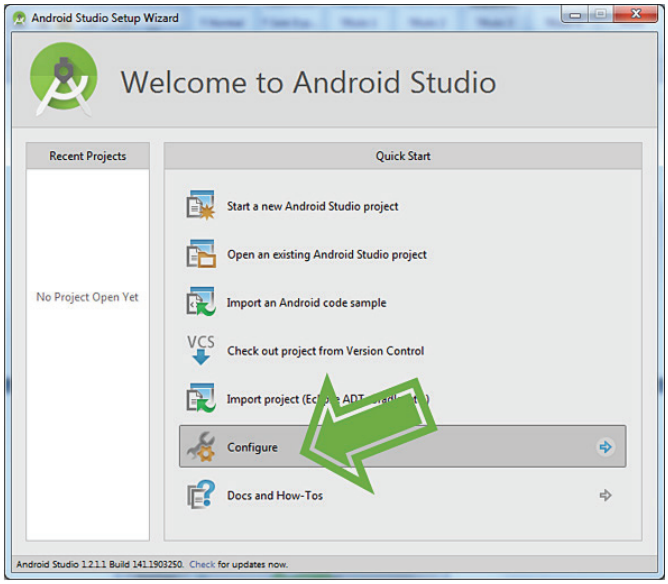


Figura 11

Selecione SDK Manager.

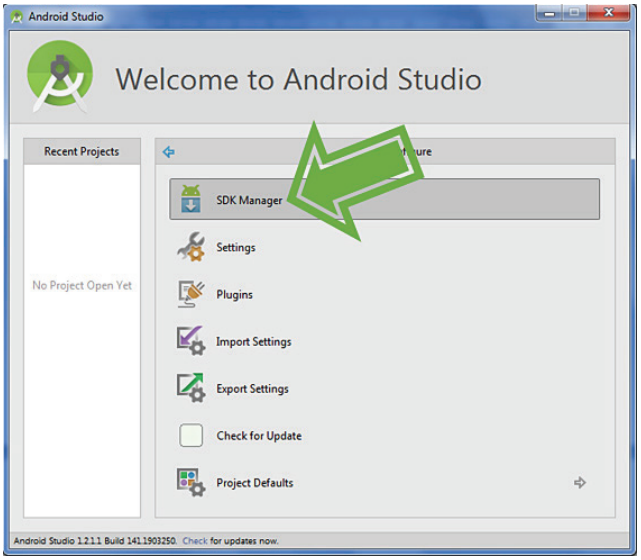


Figura 12

Ao executar o SDK Manager, adicione tudo do Android e instale as atualizações.

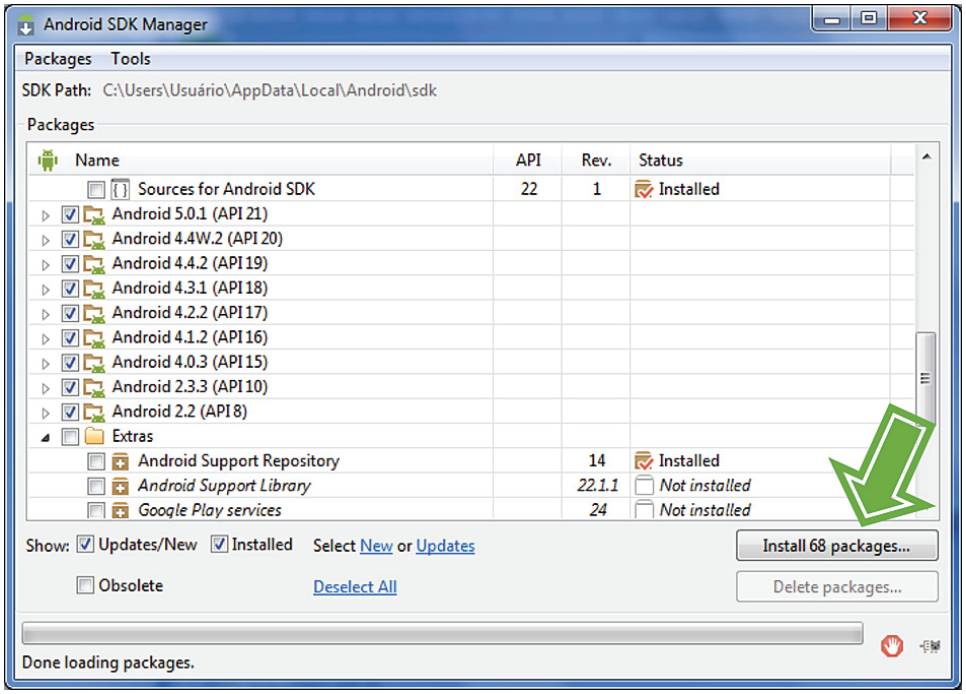


Figura 13

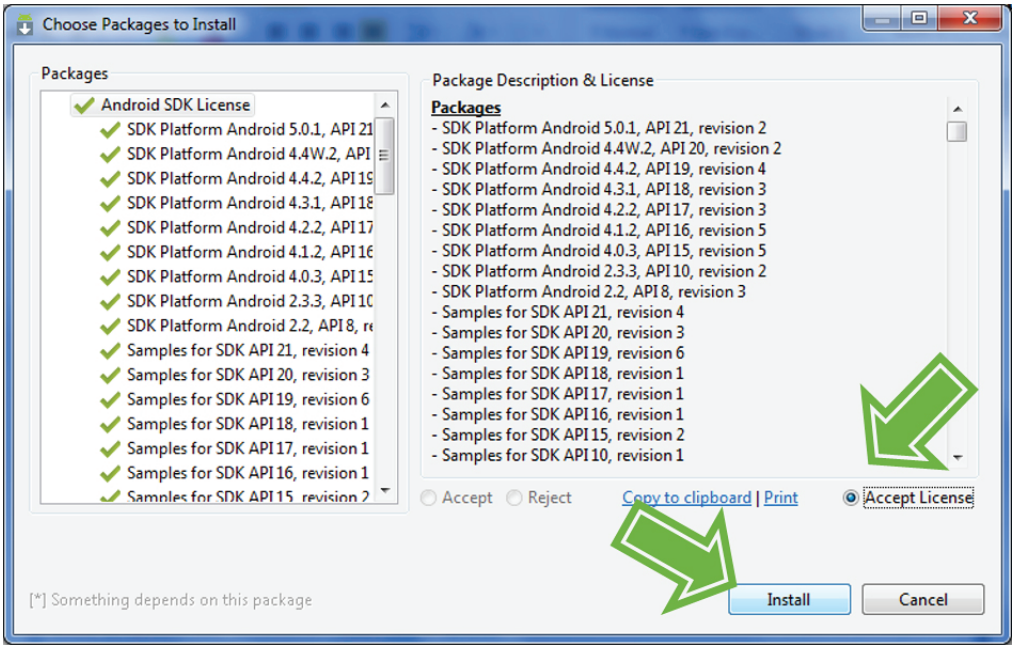


Figura 14

É necessário estar conectado na Internet para instalar esses pacotes.

Selecione todos os itens e aceite a licença e selecione *Install*.

Futuramente, você poderá adicionar mais pacotes caso necessite, utilizando o menu Tools – Android – SDK Manager.

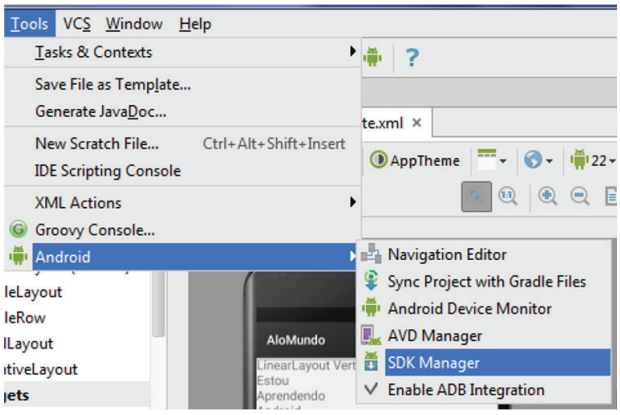


Figura 15

Criando um projeto no Android Studio.

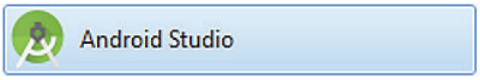


Figura 16

Execute o aplicativo:



Figura 17

Um projeto Android contém todos os arquivos que compõem o código-fonte de seu aplicativo Android. As ferramentas do Android Studio tornam mais fáceis começar um projeto novo Android com um conjunto de "diretórios padrão" do projeto e arquivos.

Nesta aula, iremos criar um novo projeto usando o Android Studio. Iremos configurar o local onde serão criados nossos projetos e o nome do nosso projeto. Para este primeiro exemplo, iremos dar o nome de AlôMundo.

Para criar um projeto siga as instruções abaixo:

1. Selecione Start a new Android Studio project.

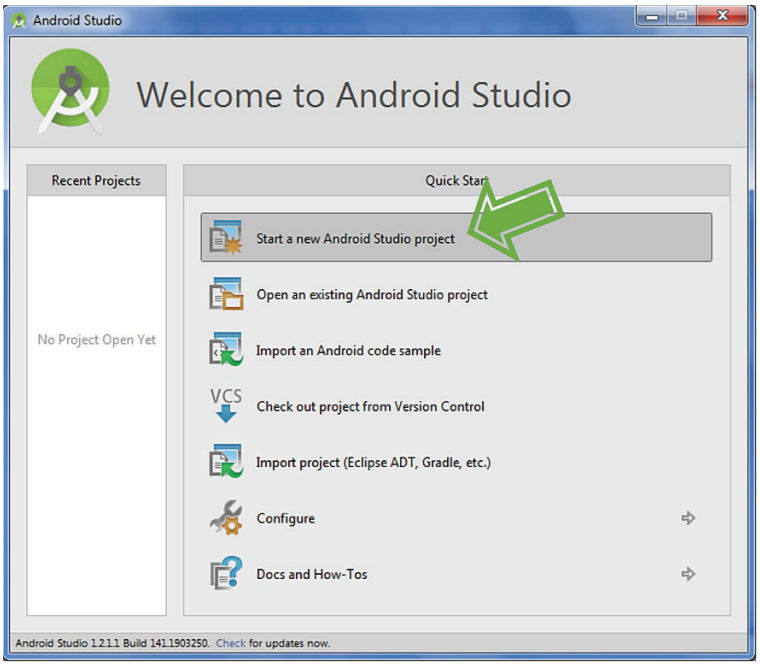


Figura 18

2. Na janela que aparece, digite a localização do projeto e no nome da aplicação conforme abaixo:

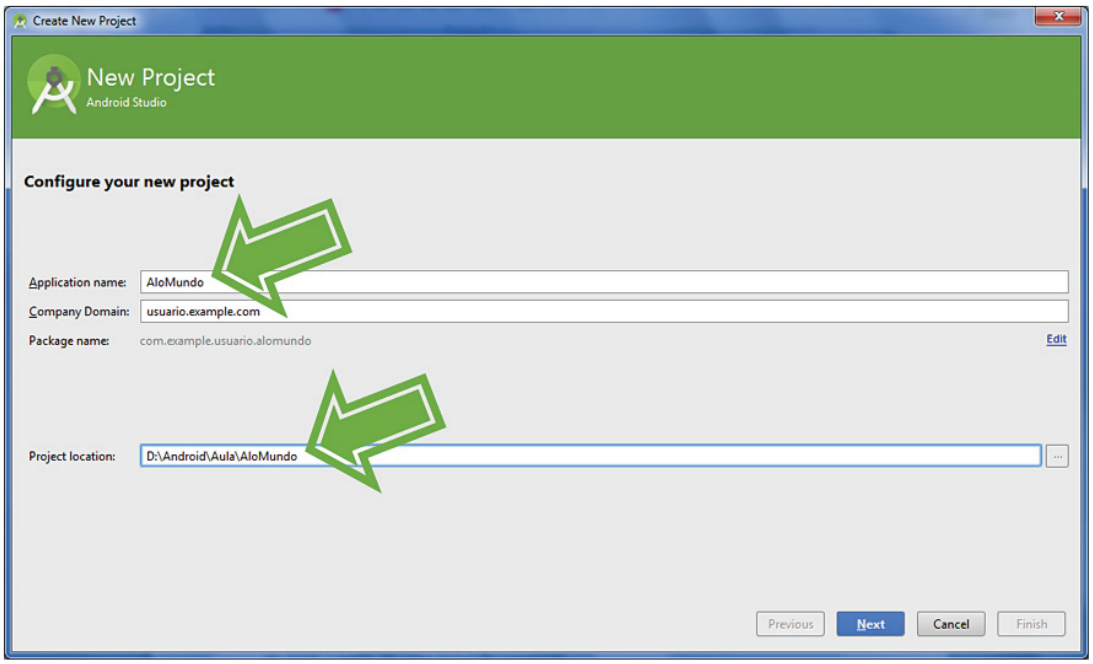


Figura 19

- **Application Name:** é o nome do aplicativo que aparece para os usuários. Para este projeto, use "AloMundo".

- **Project Location:** é o nome do diretório do projeto.
- **Company Domain:** é o namespace do pacote para o seu aplicativo (seguindo as mesmas regras de pacotes na linguagem de programação Java). O seu nome de pacote deve ser único em todos os pacotes instalados no sistema Android. Por esta razão, é geralmente melhor se você usar um nome que começa com o nome de domínio reverso da sua organização ou entidade editora. Para este projeto, você pode usar algo como "usuario.example.com". No entanto, você não pode publicar seu aplicativo no Google Play usando o "com.example" namespace.

3. Selecione projeto para Phone e a versão mínima da SDK.

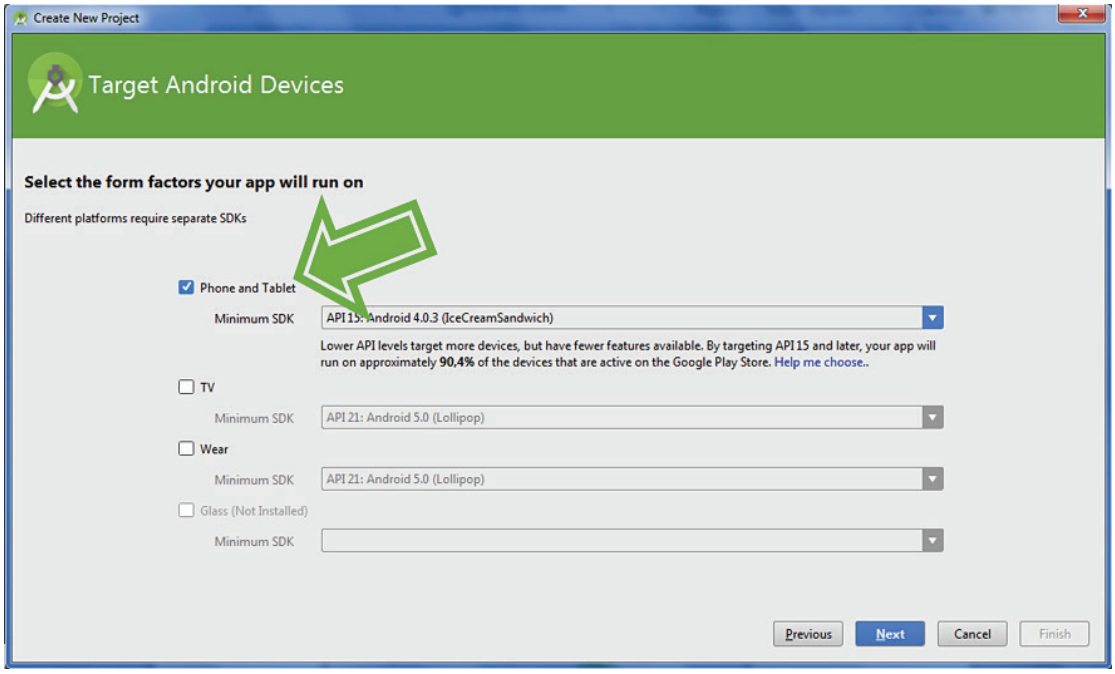


Figura 20

Mínimo Necessário SDK é a menor versão do Android que o aplicativo suporta, indicado usando a API nível. Para apoiar como dispositivo possível, você deve definir isso para a menor versão disponível que permite que seu aplicativo para fornecer seu conjunto de recursos do núcleo. Se qualquer recurso de sua aplicação só é possível em versões mais recentes do Android e que não é fundamental para a aplicação do conjunto de recursos do núcleo, você pode ativar o recurso apenas quando rodando em versões que suportam (como discutido no suporte a versões plataforma diferente). Deixe este conjunto para o valor padrão para este projeto.

4. Agora, você pode selecionar um modelo de atividade de que para começar a construir a sua aplicação. Para este projeto, selecione BlankActivity e clique em **Avançar**.



Figura 21

5. Deixe todos os detalhes em seu estado padrão e clique em **Concluir**.

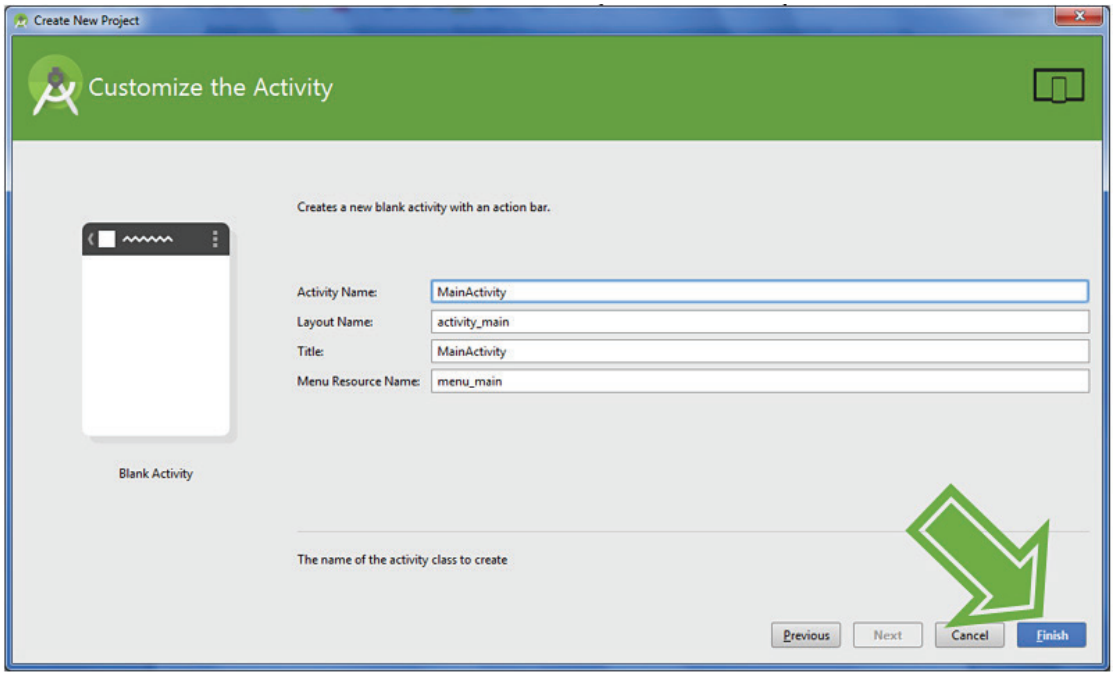


Figura 22

Entendendo o que foi gerado

Muitos arquivos foram gerados automaticamente quando criamos um projeto do Android. Para começarmos a entender o que acontece na nossa aplicação, vamos explorar um pouco a estrutura deste projeto utilizando a janela Project.

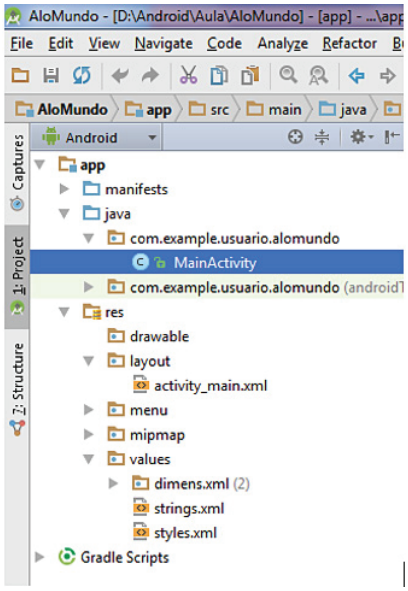


Figura 23

A classe MainActivity:

Abra o arquivo MainActivity.java (em src, no pacote com.example.usuario.olamundo). Você verá algo como:

```
MainActivity.java ×
package com.example.usuario.alomundo;

import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;


public class MainActivity extends ActionBarActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {...}

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {...}
}
```

Figura 24

A estrutura do projeto pode ser vista usando Project.

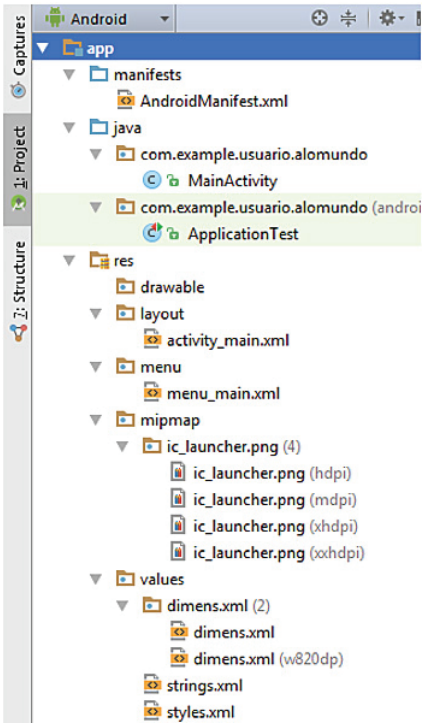


Figura 25

- **java** – contém as classes Java. Contém a classe MainActivity.
- **app/manifests/AndroidManifest.xml** – descritor da app contendo as permissões, declaração de componentes (activity, service, broadcast receiver etc.).
- **res** – contém os recursos da aplicação como imagens, layouts de telas e arquivos de internacionalização.
- **drawable** – contém as imagens da aplicação com 4 resoluções: ldpi (lowdots per inch), mdpi (medium dots per inch), hdpi (high dots per inch) e xdpi (extra high dots per inch).
- Frequentemente precisamos de imagens para complementar nossa aplicação. Entretanto, quando pensamos em dispositivos pequenos, é necessário considerar as capacidades de resolução da tela.
- Assim, um projeto Android ganha imagens nas pastas res/drawable-[hlm]dpi/, onde ldpi corresponde à baixa resolução, mdpi a média e hdpi à alta resolução.
- **layout** – contém os arquivos XML de layouts para construir as telas da aplicação.
- **menu** – contém os arquivos XML dos menus da aplicação.
- **values** – contém os arquivos XML utilizados para a internacionalização da aplicação, configuração de temas e outras configurações.

Entendendo a classe R.java

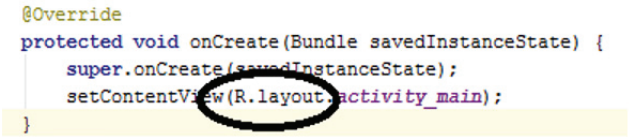


Figura 26

Existe também uma classe central e automaticamente gerada que é responsável pelo mapeamento dos elementos da view com o model e o controller. Veja que tudo o que é criado ou declarado na pasta res ganha uma representação em Java, que utilizaremos bastante para utilizar valores e referenciar itens.

Note, no entanto, que essa classe não deve ser alterada a mão. Ela é apenas um recurso que o Android disponibiliza para facilitar a referência a objetos visuais, imagens, strings etc.

Note também que nossas classes java, para acessar qualquer recurso declarado na pasta res/, fazemos uma referência aos atributos públicos desta estranha classe R. Por exemplo:

Imagens:

- R.drawable.icon

Acesso a itens da tela declarados nos XMLs de layout:

- R.id.botao
- R.id.texto

Acesso a arquivos:

- R.raw.musica
- R.raw.video

Acesso a strings:

- R.string.app_name
- R.string.hello

Veremos com frequência códigos que peguem esses componentes, strings, imagens por meio do método findViewById, para depois utilizá-los, definir listeners etc.

Essa classe R é gerada pelo plugin do Android Studio e não deve ter seus valores alterados. Ela será utilizada como referência para acessar os diversos componentes da nossa inter4face. É como uma grande coleção de identificadores, que a plataforma saberá utilizar para localizar seus valores e propriedades de maneira otimizada.

O R.java é gerado de acordo com os diversos arquivos utilizados para configuração da sua aplicação. Se houver algum erro nesses arquivos, é capaz de que o plugin não consiga regerá-lo, dessa forma propagando um erro em que todos os seus arquivos fontes. Devemos então ficar atendo a buscar o erro que está impedindo sua geração, e tomar cuidado para não importar a classe android.R, que não é a da sua aplicação, e sim uma de uso interno ao Android.

Layouts

A forma e disposição com que os elementos são apresentados na tela são de responsabilidade dos Layouts.

Um Layout é uma estratégia ou disposição para colocar os componentes dentro de uma Activity. Uma Activity poderá ser projetada como uma disposição descrita por um ou vários layouts.

Um layout poderá ser criado num arquivo XML (que é o mais frequente) ou durante a execução do programa.

Os layouts mais comuns para organizar elementos de interface na tela são: LinearLayout, RelativeLayout e TableLayout. Iremos falar mais sobre esses tipos de layouts mais adiante.

Veja arquivo *activity_main.xml* em *res/layout* no Project.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools" android:layout_width="match_parent"
    android:layout_height="match_parent" android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:paddingTop="16dp"
    android:paddingBottom="16dp" tools:context=".MainActivity">

    <TextView android:text="Hello world!" android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

</RelativeLayout>
```

Figura 27

Vamos substituir o TextView acima com os dados abaixo para compreender a utilização de strings:

```
<RelativeLayout xmlns:android=”http://schemas.android.com/
apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools”
android:layout_width=”match_parent”
    android:layout_height=”match_parent”
android:paddingLeft=”@dimen/activity_horizontal_margin”
    android:paddingRight=”@dimen/activity_horizontal_margin”
    android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
tools:context=”.MainActivity”>
```

```
        <TextView
            android:layout_width=”fill_parent”
            android:layout_height=”wrap_content”
            android:text=”@string/hello_world”
            />
    </RelativeLayout>
```

A tag TextView tem altura determinada apenas necessário para mostrar seu conteúdo, configurada como wrap_content.

Agora, repare na linha em destaque: *@string/hello_world*. Ela define o texto que aparecerá na tela quando a aplicação for executada. Perceba que ela é apenas uma chave para algum texto. Esse texto está definido em *res/values* dentro do arquivo *strings.xml*.

```
<resources>
    <string name="app_name">AloMundo</string>

    <string name="hello_world">Hello world!</string>
    <string name="action_settings">Settings</string>
</resources>
```

Figura 28

Essa é uma ideia simples, mas que facilita na internacionalização da sua aplicação, podendo, assim, ser introduzida para outras línguas, bastando criar outros arquivos de *values* com as traduções.

Selecionando o modo gráfico, notaremos como ficou o layout.

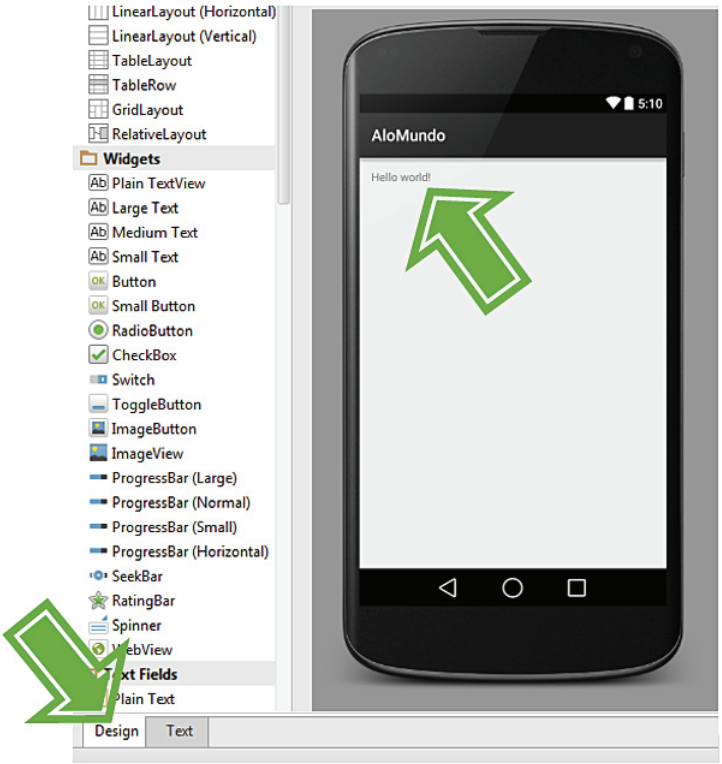


Figura 29

Configuração do emulador

Depois de instaladas a plataforma e a imagem de cada versão do Android, o próximo passo é criar o(s) emulador(es) para testarmos nossos aplicativos.

Acesse agora o menu **Tools – Android – AVD Manager**.

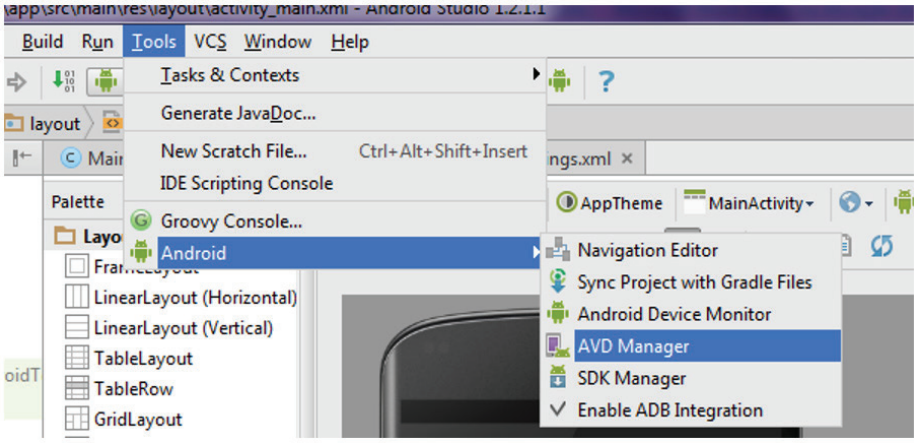


Figura 30

Clique no botão **Create a virtual device** para adicionarmos um novo **Emulador**.

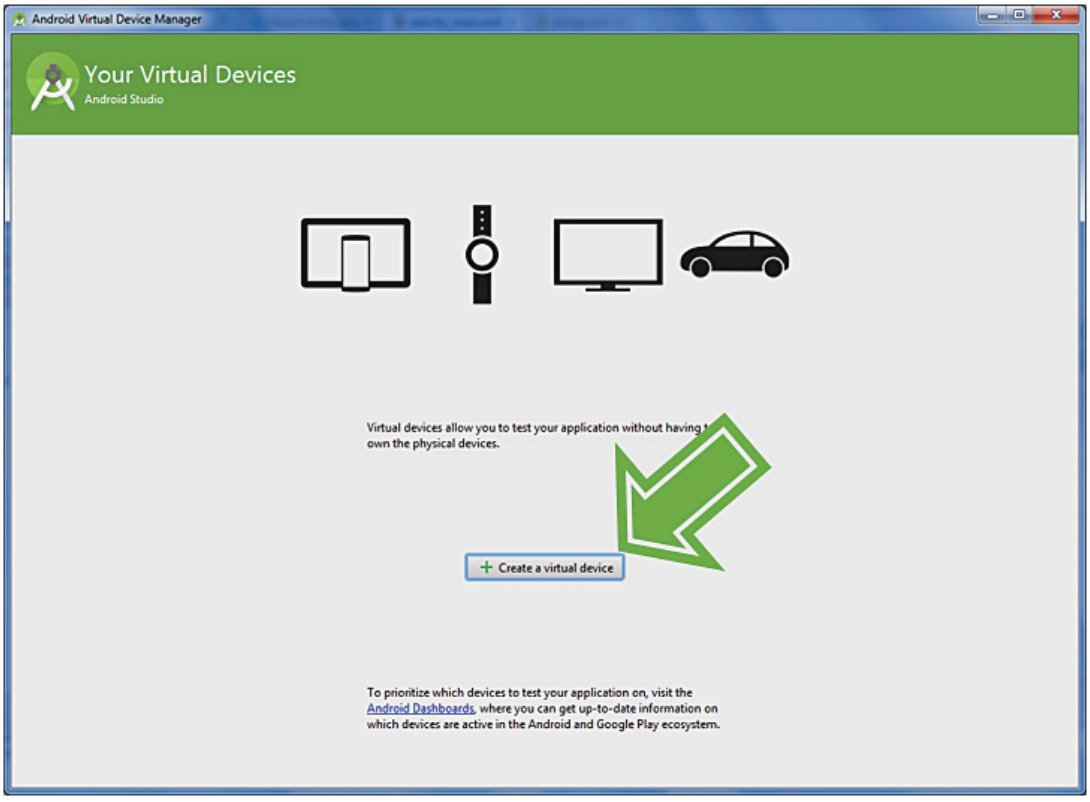


Figura 31

Na tela de escolha do novo AVD, podemos utilizar várias resoluções, e para nossas aulas utilizaremos o Nexus 4 com resolução de 768 x 1280 pixels, conforme tela abaixo.

Selecione o botão Próximo.

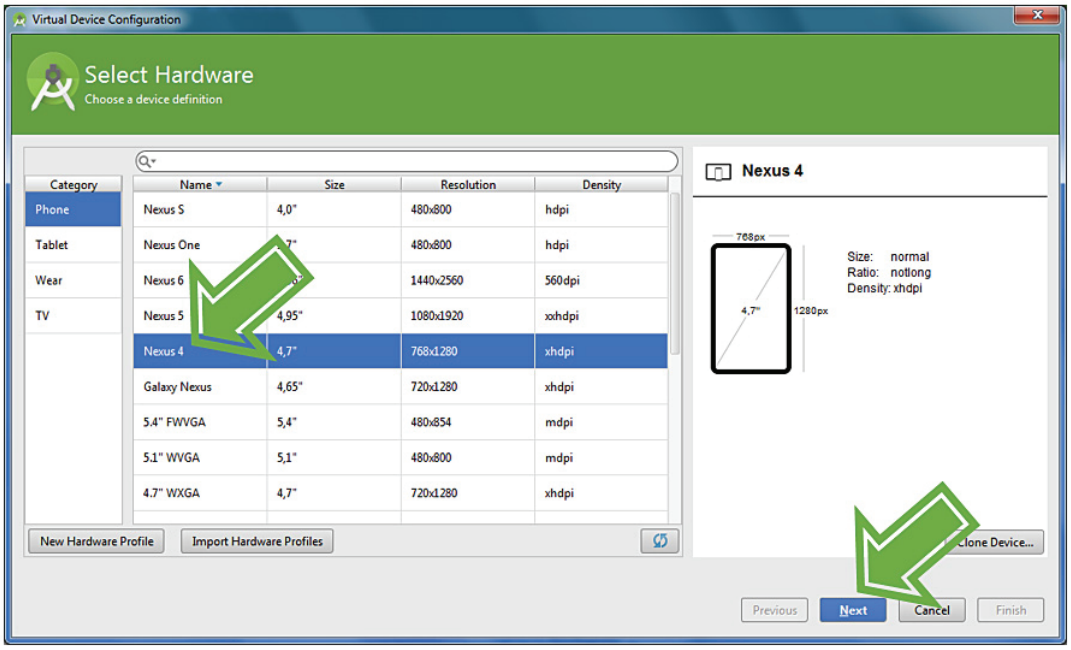


Figura 32

Nossa próxima configuração é a versão do Android, e para nossa aula iremos utilizar a versão KitKat, ou seja, Android 4.4.2. Caso a mesma não esteja instalada, selecione o Donwload para que o mesmo possa fazer parte do nosso projeto.

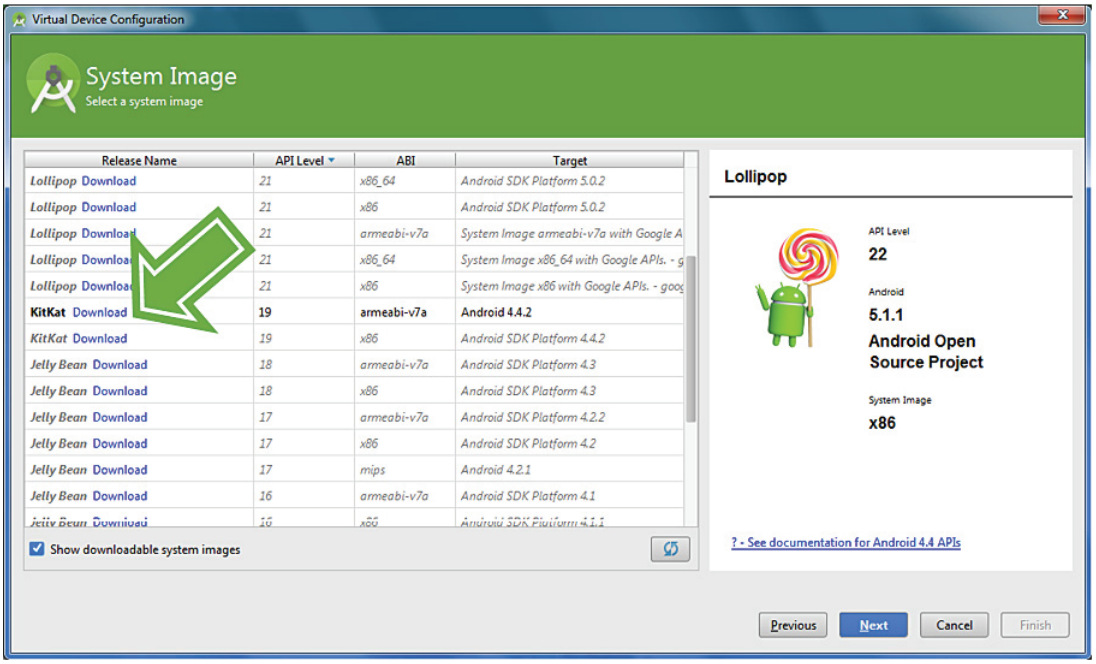


Figura 33

Após o Download, selecione a versão do Android 4.4.2 (KitKat) e selecione o botão Próximo.

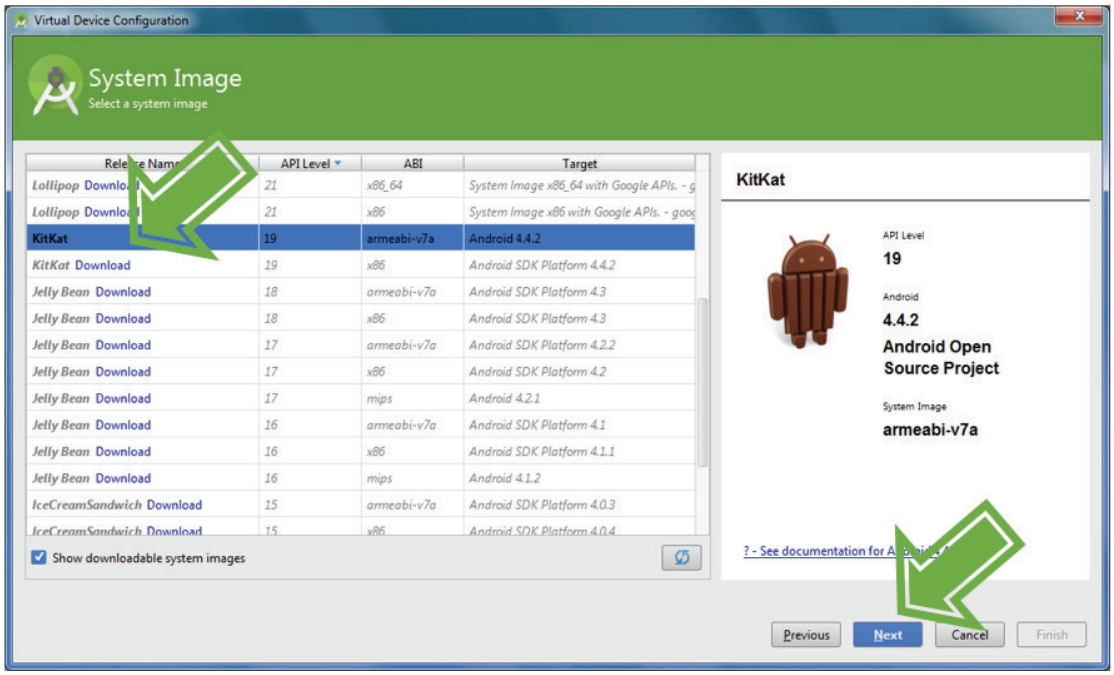


Figura 34

Na próxima tela, iremos finalizar nossa configuração.

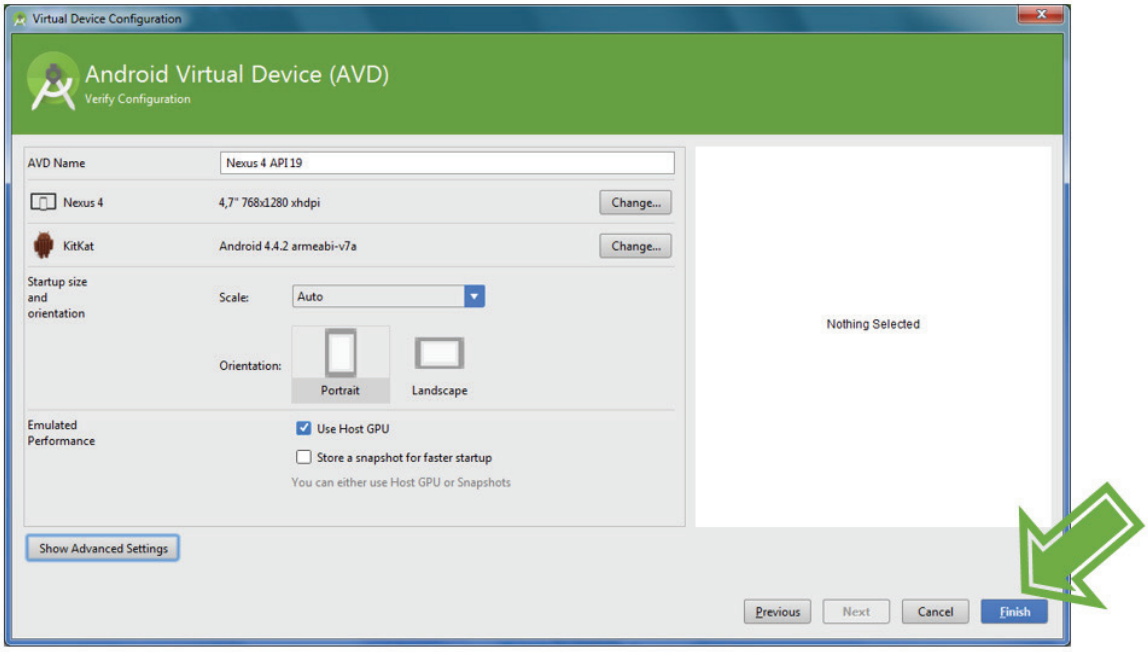


Figura 35

Pronto, nosso emulador já pode ser executado.

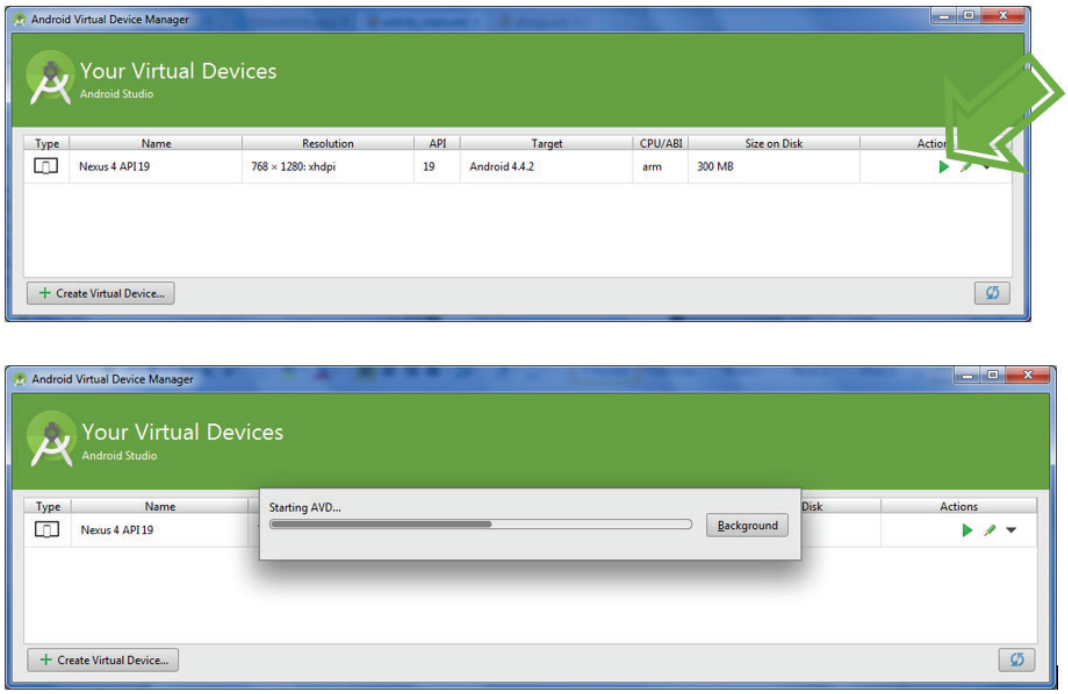


Figura 36

Pronto, nosso emulador já está operando e já pode receber a nossa primeira aplicação.

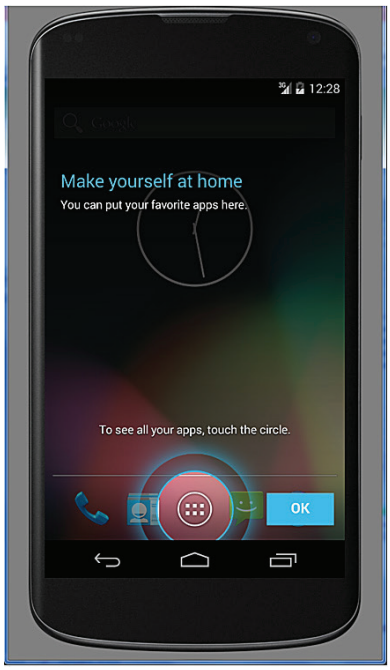


Figura 37

Agora, iremos executar nossa aplicação AloMundo. Selecione a figura abaixo para executar nossa aplicação.

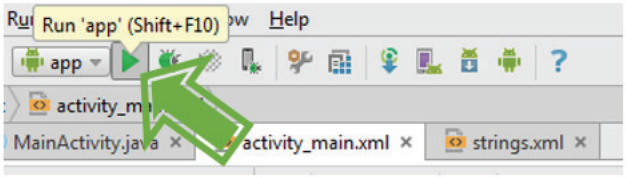


Figura 38

Ou caso prefira, utilize o menu Run, Run 'app'.

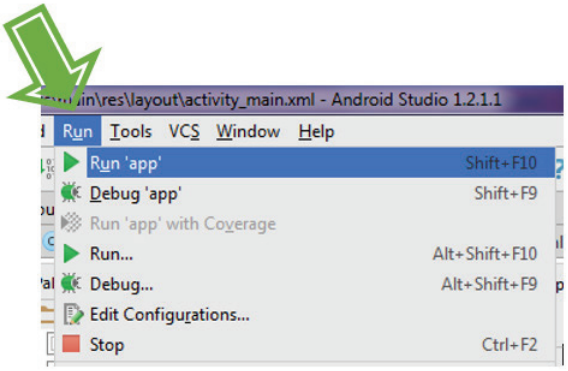


Figura 39

Nesta próxima janela, será apresentado o emulador no qual iniciamos. Portanto, selecione o botão OK.

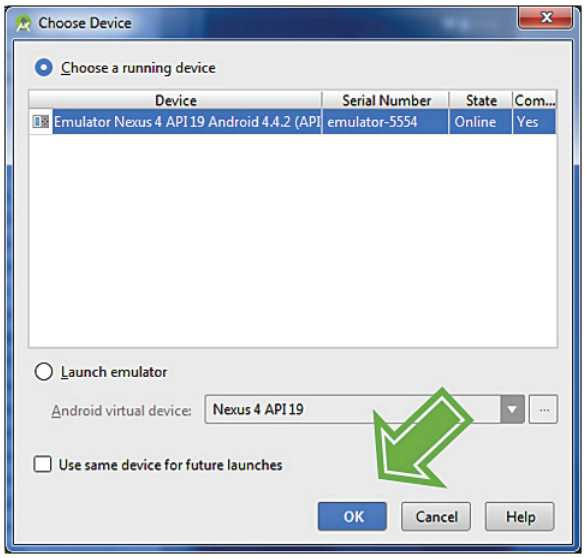


Figura 40

O sistema irá compilar a aplicação e, após alguns segundos, será aberta nossa aplicação no emulador que tínhamos iniciado.

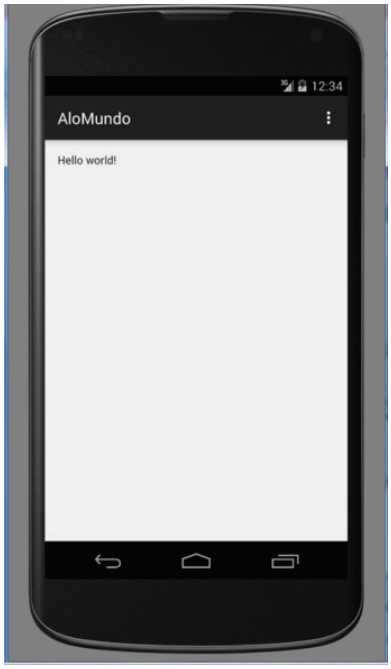


Figura 41

Caso queira parar a aplicação, selecione o item abaixo Android e o item Terminate Application.

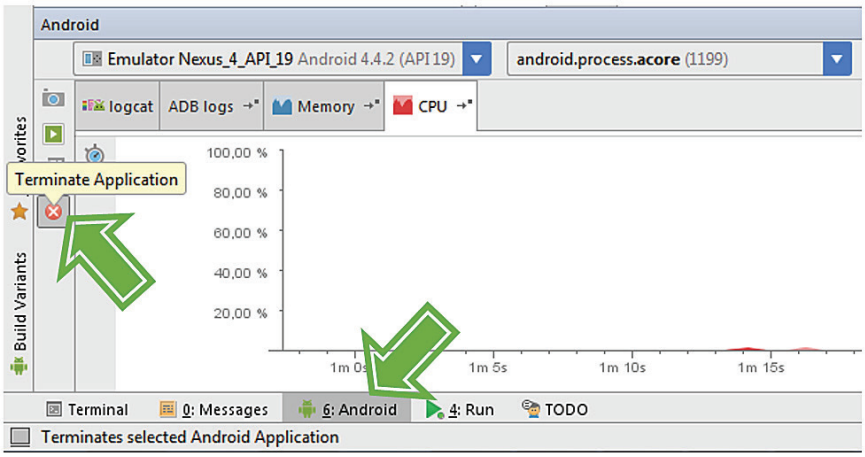


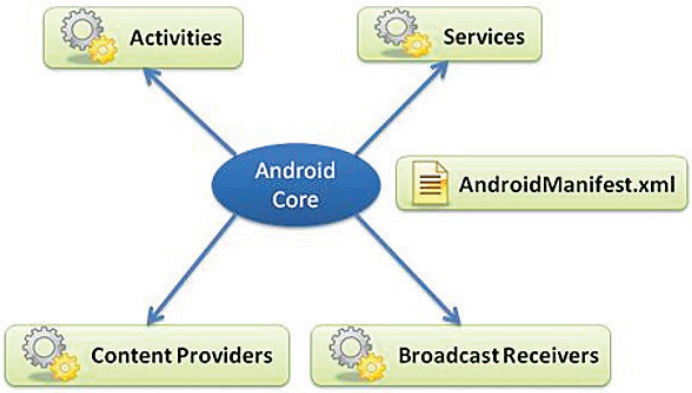
Figura 42

Conceitos fundamentais

Vamos entender agora os componentes e seus ciclos de vida para que possamos montar um sistema robusto, expansível e com uma performance aceitável para o usuário.

Componentes de uma Aplicação Android

Desenvolver aplicações para Android significa compor uma série de componentes para que o objetivo final da aplicação seja atingido. A Figura abaixo mostra estes componentes.



Componentes de uma aplicação Android

Figura 43

Activities são as representantes das telas da aplicação. Associada a uma activity normalmente existe uma view, que define como será feita a exibição visual para o usuário. As activities são responsáveis por gerenciar os eventos de tela e também coordenam o fluxo da aplicação.

Uma Activity (atividade) é um objeto individual que poderá receber o foco e que poderá ser uma janela a tela inteira (que é o mais frequente), uma janela flutuante ou uma janela embutida dentro de outra janela. Activity é como se fosse o executável da sua aplicação, onde deve estender a classe Activity, pois será a maneira de o framework saber o que chamar no seu executável.

O método onCreate é chamado quando uma Activity é executada e ele faz a chamada no sistema. As classes que estendem Activity, de uma forma mais abrangente, interagem tanto com usuários como com serviços ou intenções.

Toda Activity tem um ciclo de vida representado na figura abaixo. Os métodos chamados serão os métodos que sobrescreveremos quando criarmos nossas próprias Activities.

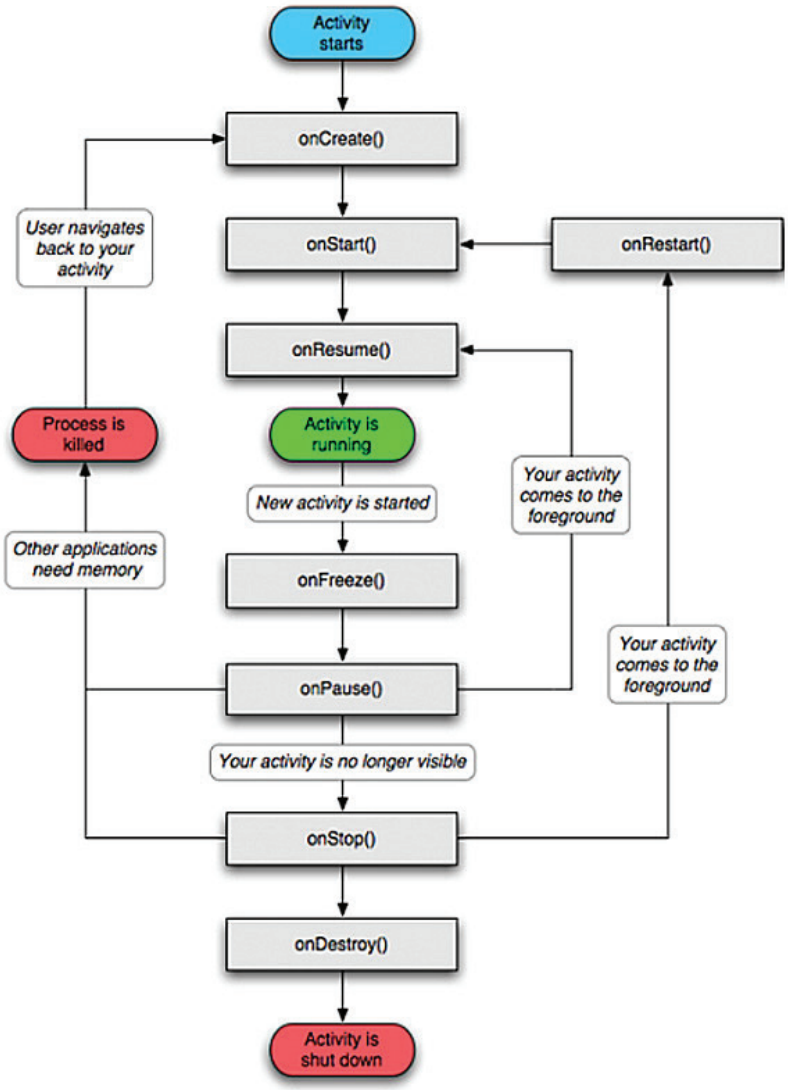


Figura 44

onCreate()

É a primeira função a ser executada em uma Activity. Geralmente, é a responsável por carregar os layouts XML e outras operações de inicialização. É executada apenas uma vez.

onStart()

É chamada imediatamente após a onCreate() – e também quando uma Activity que estava em background volta a ter foco.

onResume()

Assim como a onStart(), é chamada na inicialização da Activity e também quando uma Activity volta a ter foco. Qual a diferença entre as duas? A onStart() só é chamada quando a Activity não estava mais visível e volta a ter o foco, a onResume() é chamada nas "retomadas de foco".

onPause()

É a primeira função a ser invocada quando a Activity perde o foco (isso ocorre quando uma nova Activity é iniciada).

onStop()

Só é chamada quando a Activity fica completamente encoberta por outra Activity.

onDestroy()

A última função a ser executada. Depois dela, a Activity é considerada "morta" – ou seja, não pode mais ser relançada. Se o usuário voltar a requisitar essa Activity, um novo objeto será construído.



Os Services são códigos que executam em segundo plano. Normalmente, são utilizados para tarefas que demandam um grande tempo de execução.

Os Content Providers (provedores de conteúdos) são a maneira utilizada pela plataforma para compartilhar dados entre as aplicações que executam no dispositivo. Um exemplo bem claro disto é a aplicação de gerenciamento de contatos do Android, que é nativa. Aplicações desenvolvidas por terceiros podem utilizar um content provider a fim de ler os contatos armazenados no dispositivo de forma simples.

Os Broadcast Receivers são componentes que ficam "escutando" a ocorrência de determinados eventos, que podem ser nativos ou disparados por aplicações. Uma aplicação pode, por exemplo, utilizar um broadcast receiver para ser avisada quando o dispositivo estiver recebendo uma ligação e, com base nessa informação, realizar algum tipo de processamento.

Junto a esses componentes, existe o arquivo de manifesto AndroidManifest.xml. Ele é obrigatório e único para cada aplicação. É nele que são feitas as configurações gerais da aplicação e dos componentes que fazem parte dela. E, juntando tudo isso, existe a figura do **Android Core**, que, na verdade, não é um componente específico, mas sim a plataforma Android propriamente dita.

É ele quem proporciona a interação entre os componentes e as aplicações e torna possível a execução do código.

Componentes de tela:

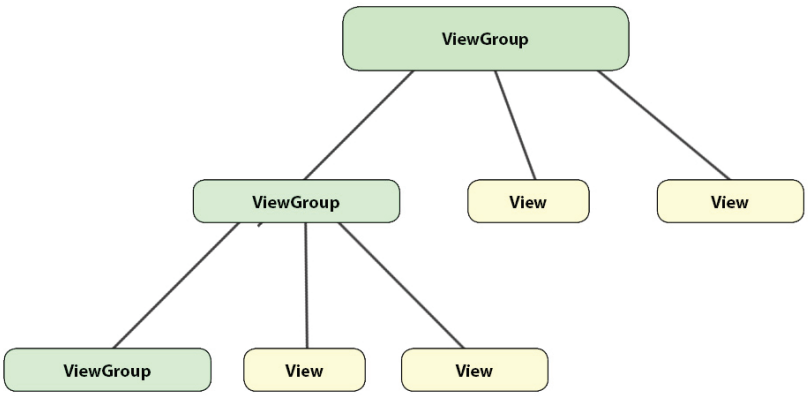
Todas as telas do Android são feitas em um arquivo XML, que fica dentro de res/layout, onde é configurado os detalhes de cada componente. Um componente é um item gráfico.

Dentro do XML, cada componente declarado poderá ser manipulado em código Java. Todos esses componentes são filhos de android.view.View. Esses Views serão futuramente agrupados dentro de ViewGroup. Não coincidentemente, ViewGroup é filha de View, formando o composite pattern, que aparece também no Swing, onde Container é filha de Component.

View (vista) é a classe básica que permite construir componentes de interface. Uma View ocupa uma área retangular na tela e é responsável por desenhar e atender eventos, caso seja necessário.

Quando necessário, podemos definir uma classe derivada de View, que implemente o método (evento) OnDraw (com cabeçalho @Override protected void onDraw(Canvas c)) para desenhar na vista. O método invalidate() provocará a execução do evento mencionado. Essa vista poderá ocupar uma parte da tela do aparelho, compartilhando a tela com outros elementos.

Uma ViewGroup pode conter Views e inclusive ViewGroups conforme figura abaixo:



Hierarquia de componentes de tela.

Figura 45

Algumas pessoas irão chamar esses componentes de widget, mas há um outro conceito no android, que é uma pequena aplicação que pode rodar dentro da home do dispositivo móvel, também chamado de widget.

Os componentes de interface mais frequentes são criados com as classes TextView, EditText, Button, RadioButton, RadioGroup, CheckBox etc., todas derivadas direta ou indiretamente da classe View.

As classes de layouts LinearLayout, RelativeLayout, AbsoluteLayout, TableLayout e FrameLayout são derivadas indiretamente da classe View.

A seguir, alguns dos componentes mais utilizados, suas propriedades e alguns listeners:

TextView

Serve para escrevermos um texto na tela do Android, como um label.

```
 <?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/
android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Hello Views!"
        android:textSize="28px"
        android:textColor="#12FF88"
        android:textStyle="italic|bold"
    />
</LinearLayout>
```

O TextView é usado para exibir texto para o usuário, assim como o JLabel do Swing. Podemos controlar sua aparência e tamanho pelo XML, basta modificar os atributos na tag TextView, alguns desses atributos são:

- **android:text:** valor será o texto do componente;
- **android:textColor:** o valor da cor no formato '#RRGGBB' em hexadecimal, é a cor do texto;
- **android:textSize:** tamanho da fonte, deve ser indicado em px, ex: '12px';
- **android:textStyle:** style do texto (bold, italic);
- **android:background:** uma cor no formato #RRGGBB ou um drawable.

Alguns atributos são valores pré-defenidos como os do android:textStyle, mesmo assim, ainda é possível usar mais de um valor no atributo, é aqui que entra o operador pipe " | ".

Por exemplo, se quisermos o texto em negrito usamos "bold", em itálico usamos o valor "italic", mas se quisermos que o texto seja negrito+itálico, podemos usar o pipe juntando os dois: "bold | italic".



Figura 46

EditText

Enquanto o TextView exibe texto como um JLabel, quem oferece ao usuário um campo para ele colocar texto é o EditText (assim como o JTextField) que é filho de TextView.

Este componente não declara nenhum atributo XML novo, mas usa os do TextView, a diferença é que ele realmente permite edição.

Para tornar ele editável ou não, usamos o atributo android:editable com true ou false (true é o default). O TextView mesmo com true não permite edição.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/
android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Hello Views!"
        android:textSize="28px"
        android:textColor="#12FF88"
        android:textStyle="italic|bold" />
    <EditText android:id="@+id/editView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="..."
        android:textSize="28px"
        android:textColor="#12FF88"
        android:textStyle="italic|bold" />
</LinearLayout>
```

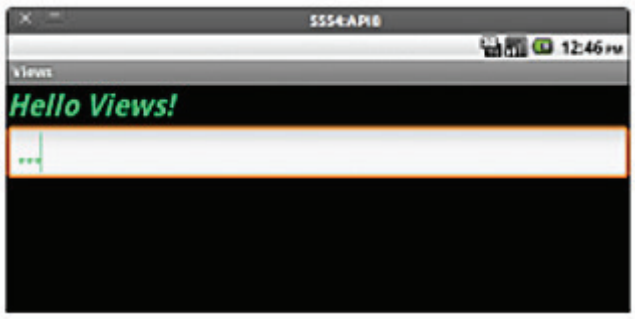


Figura 47

Button

É um simples botão com algo escrito assim como o JButton. É por meio dele que o usuário sabe que alguma ação será realizada quando clicar.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/
android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Hello Views!"
        android:textSize="28px"
        android:textColor="#12FF88"
        android:textStyle="italic|bold" />
   <EditText android:id="@+id/editView"
        android:layout_width="match_parent"
       android:layout_height="wrap_content"
        android:text="..."
        android:textSize="28px"
        android:textColor="#12FF88"
        android:textStyle="italic|bold" />
   <Button android:id="@+id/button"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Click!"
        android:textSize="28px"
        android:textColor="#12FF88"
        android:textStyle="italic|bold" />
</LinearLayout>
```

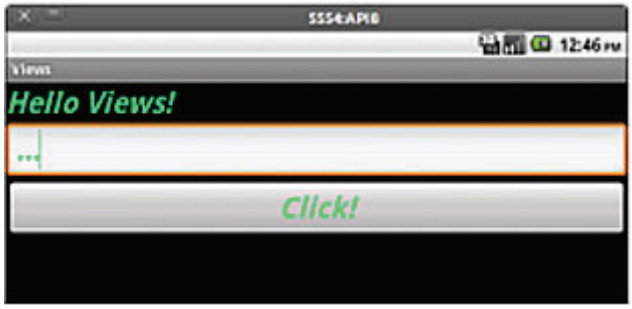


Figura 48

CheckBox

É um botão que permite ao usuário alternar entre as opções marcado/desmarcado.

Ele também estende TextView, e o valor atribuído em 'android:text' é o texto visivel ao lado do CheckBox.

ToggleButton

ToggleButton são botões como o CheckBox, mas indicam marcado/desmarcado com uma 'luz' invés do efeito de desenho de um "tick". O atributo 'android:text' não tem efeito nesse componente, por padrão o texto é alternado entre ON e OFF, sendo possível usar os atributos como color, style, size e etc. para modificar a aparência desse texto.

Esse componente tem dois atributos a mais para sua tag.

- **android:textOn:** o texto que será exibido quando estado for ON;
- **android:textOff:** o texto que será exibido quando estado for OFF.

RadioButton

São botões como o CheckBox que permitem marcá-lo, mas não é possível desmarcá-lo. Para se obter o efeito de seleção única, é necessário adicioná-los a um RadioGroup.

E o nosso XML final:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout android:orientation="vertical"
    xmlns:android="http://schemas.android.com/apk/res/
android"
    android:layout_width="match_parent" android:layout_
height="match_parent">
   <TextView android:id="@+id/listView" android:layout_
width="match_parent"
        android:layout_height="wrap_content" android:-
text="Hello Views!"
        android:textSize="28px" android:textCol-
or="#12FF88" android:textStyle="italic|bold" />
    <EditText android:id="@+id/textView" android:layout_
width="match_parent"
        android:layout_height="wrap_content" android:-
text="..."
        android:textSize="28px" android:textCol-
or="#12FF88" android:textStyle="italic|bold" />
    <Button android:id="@+id/button" android:layout_
width="match_parent"
        android:layout_height="wrap_content" android:-
text="Click!"
        android:textSize="28px" android:textCol-
or="#12FF88" android:textStyle="italic|bold" />
    <CheckBox android:id="@+id/check" android:layout_
width="match_parent"
        android:layout_height="wrap_content" android:-
text="Check it!"
```

```
        android:textSize="28px" android:textCol-
or="#12FF88" android:textStyle="italic|bold" />
    <ToggleButton android:id="@+id/tbutton"
        android:layout_width="match_parent" android:lay-
out_height="wrap_content"
        android:textOn="Ligado" android:textOff="De-
sligado" android:textSize="28px"
        android:textColor="#12FF88" android:text-
Style="italic|bold" />
    <RadioGroup android:layout_width="match_parent"
        android:layout_height="wrap_content">
        <RadioButton android:id="@+id/r1button"
            android:layout_width="match_parent" an-
droid:layout_height="wrap_content"
            android:text="Opção 1" android:text-
Size="28px" android:textColor="#12FF88"
            android:textStyle="italic|bold" />
        <RadioButton android:id="@+id/r2button"
            android:layout_width="match_parent" an-
droid:layout_height="wrap_content"
            android:text="Opção 2" android:text-
Size="28px" android:textColor="#12FF88"
            android:textStyle="italic|bold" />
    </RadioGroup>
</LinearLayout>
```

Note que todos eles têm os atributos do TextView, tornando fácil personalizá-los.

E o resultado final da nossa aplicação é como a seguinte:

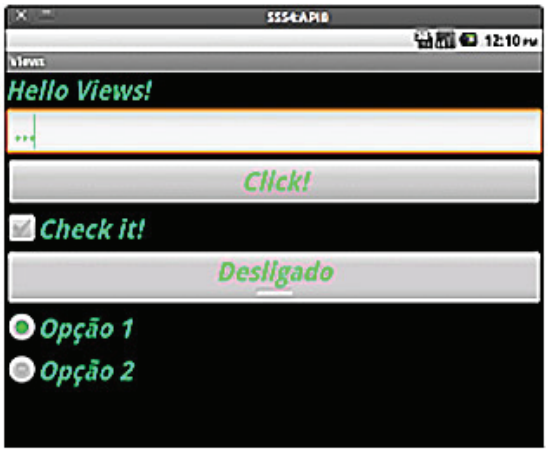


Figura 49

ImageView

O ImageView, como o próprio nome diz, é um componente que nos permite trabalhar com imagens.

Colocar imagens no Android fica muito simples com a sua pasta drawable, pois se ela não existir, ele simplesmente não mapeia no R.java.

```
<ImageView
        android:id=”@+id/imageView1”
        android:layout_width=”fill_parent”
        android:layout_height=”fill_parent”
        android:src=”@drawable/ic_launcher” />
```

Como já vimos, deve haver a imagem dentro de *res*/drawable, e a partir do Android 1.6, essa pasta possibilita 3 versões de resoluções diferentes.

Conhecendo melhor os tipos de layout

Quando definimos um arquivo de layout, jogamos dentro dele todos os componentes que gostaríamos de ter na tela, conforme estudado anteriormente. Uma questão que fica é: como esses componentes estarão dispostos na tela do dispositivo? Apesar de eles possuírem diretivas em relação a sua altura e largura, quem vem primeiro? Quem estará próximo do outro? Como definir com mais exatidão esse posicionamento?

O Android possui classes filhas de ViewGroup, sendo que algumas agem como layout managers. Eles definirão como seu layout será aplicado aos diversos componentes (Views) contidos nele. É comum misturar diferentes layouts em uma mesma tela para obter um resultado mais agradável visualmente.

Vamos estudar agora os diversos tipos de layout.

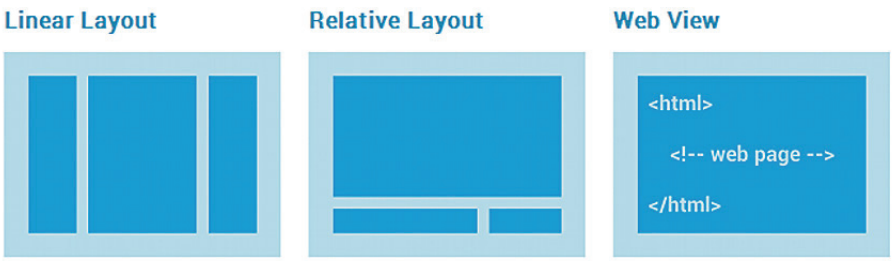


Figura 50

LinearLayout

LinearLayout é um dos layouts mais simples. Ele organiza os elementos dentro dele de forma linear, podendo ser de maneira horizontal ou vertical.

Para auxiliar nossos exemplos, iremos criar outro arquivo de layout. Com o botão direito do mouse em layout, selecione New – XML – Layout XML File.

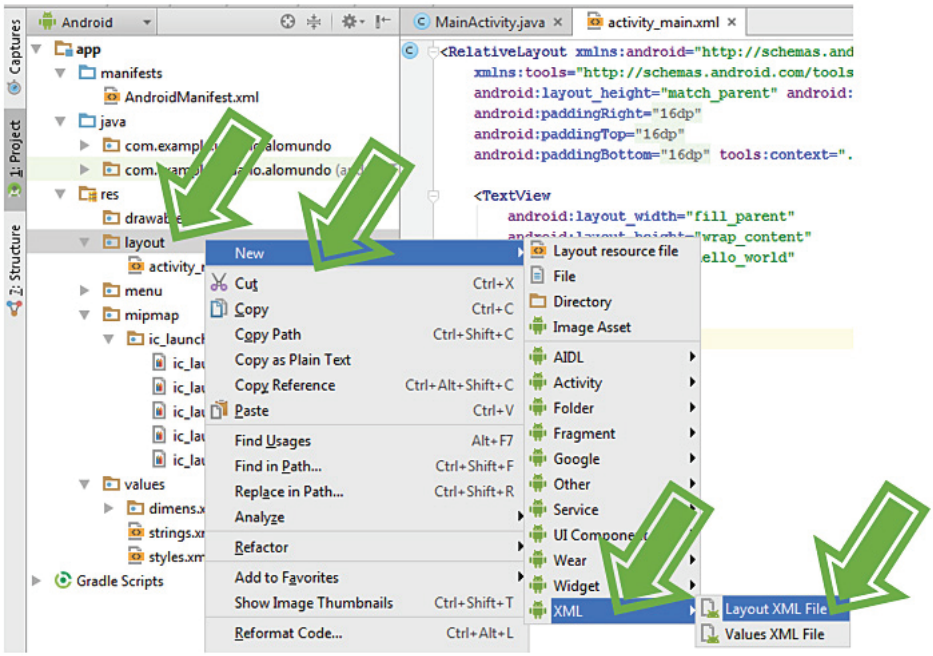


Figura 51

Agora, iremos dar um nome para nosso novo layout. Daremos o nome de layout_linear.

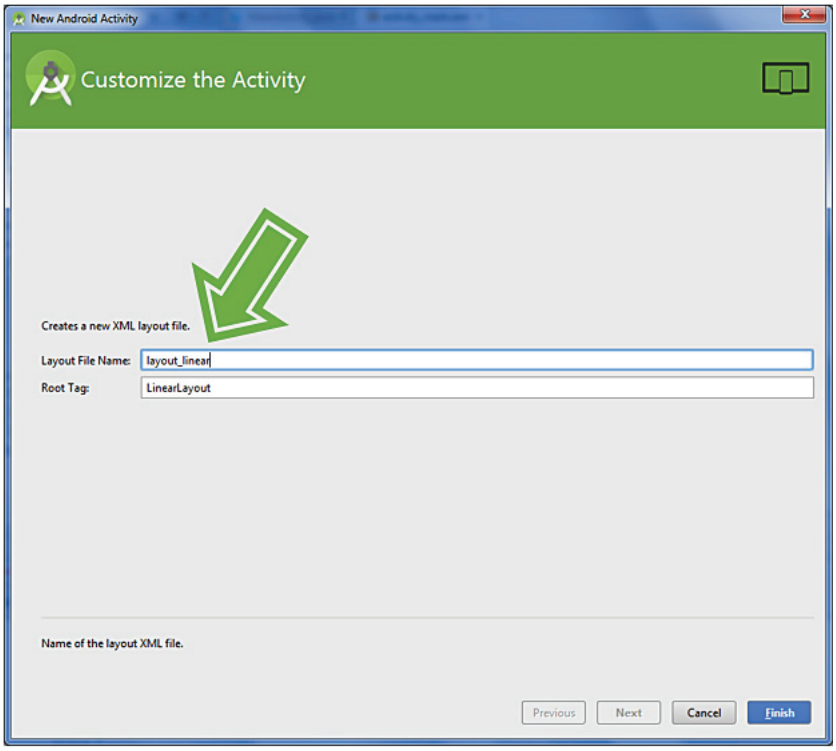


Figura 52

Abaixo, o código gerado:

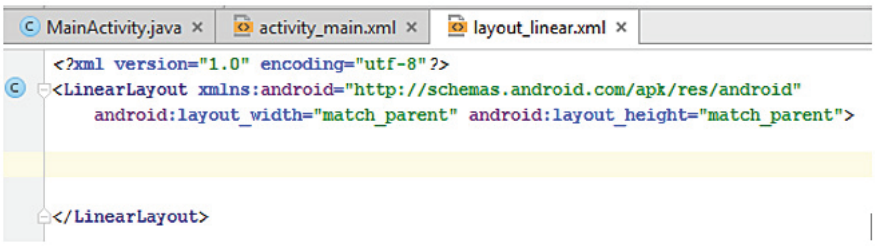


Figura 53

Agora, iremos substituir o código gerado pelo descrito abaixo:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/
apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="LinearLayout Vertical"
        android:textSize="20sp" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Estou"
        android:textSize="20sp" />

    <TextView
        android:id="@+id/textView3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Aprendendo"
        android:textSize="20sp" />

    <TextView
        android:id="@+id/textView4"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Android"
        android:textSize="20sp" />

    <ImageView
        android:id="@+id/imageView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@mipmap/ic_launcher" />
```

Orientação pode ser vertical ou horizontal

Componentes de tela

```
        <ImageView
            android:id="@+id/imageView2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:src="@mipmap/ic_launcher" />
    </LinearLayout>
```

Mude para o modo gráfico e veja como ficou:

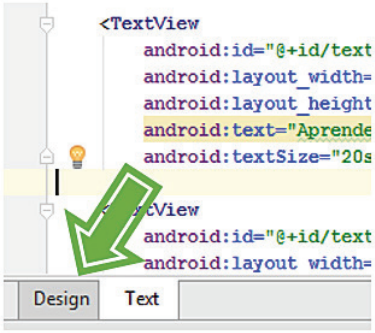


Figura 54

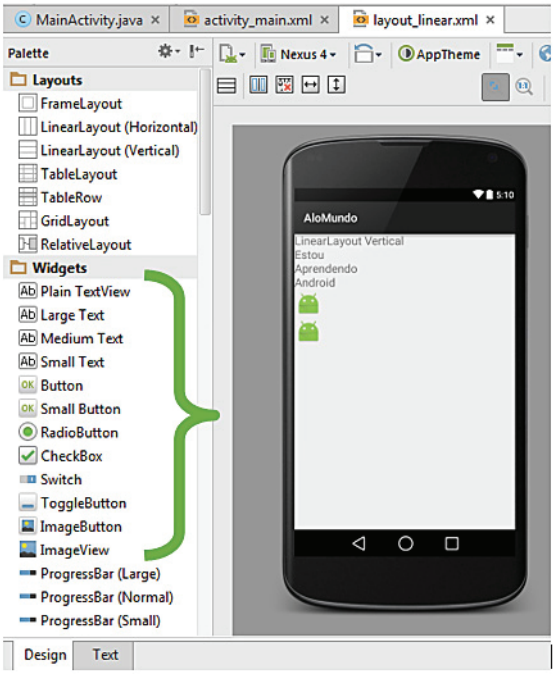


Figura 55

Caso você queira praticar mais, utilize os Widgets da janela ao lado e arraste para o dispositivo para verificar como ficará.

Troque também a orientação do layout para horizontal e veja como ficará a disposição dos objetos. Podemos definir também a estratégia para largura e altura dos itens: fill_parent ou wrap_content (fill-preencher; wrap-embrulhar, enrolar).

Podemos, também, alterar uma configuração interessante de cada componente: é o android:layout_gravity, que, permite configurar os componentes em vários locais da tela, direita, esquerda e centro.

Substitua o layout acima pelo abaixo e veja o resultado:



Figura 56

```
<?xml version=”1.0” encoding=”utf-8”?>
<LinearLayout xmlns:android=”http://schemas.android.com/apk/
res/android”
    android:layout_width=”match_parent”
    android:layout_height=”match_parent”
    android:orientation=”vertical” >

    <TextView
        android:id=”@+id/textView1”
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:layout_gravity=”left”
        android:text=”Estou” android:textSize=”22sp”/>
    <TextView
        android:id=”@+id/textView2”
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:layout_gravity=”center”
        android:text=”Aprendendo” android:textSize=”22sp”/>
    <TextView
        android:id=”@+id/textView3”
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:layout_gravity=”right”
        android:text=”Android” android:textSize=”22sp”/>

    <ImageView
        android:id=”@+id/imageView1”
        android:layout_width=”wrap_content”
```

```
            android:layout_height="wrap_content"
            android:layout_gravity="right"
            android:src="@mipmap/ic_launcher" />
        <ImageView
            android:id="@+id/imageView2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:src="@mipmap/ic_launcher" />

        <ImageView
            android:id="@+id/imageView3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="left"
            android:src="@mipmap/ic_launcher" />

    </LinearLayout>
```

TableLayout

Neste tipo de layout, colocamos os itens em linhas e colunas. Atributos comuns são: padding (espaçamento da célula) e gravity (alinhamento). Veja android:stretchColumns="0,1" (colunas a serem "esticadas"; usar * para todas).

A unidade utilizada é dip (Density-independent pixel), veja mais detalhes em:
http://developer.android.com/guide/topics/ui/layout/grid.html
http://developer.android.com/guide/practices/screens_support.html.

Exemplo de TableLayout:

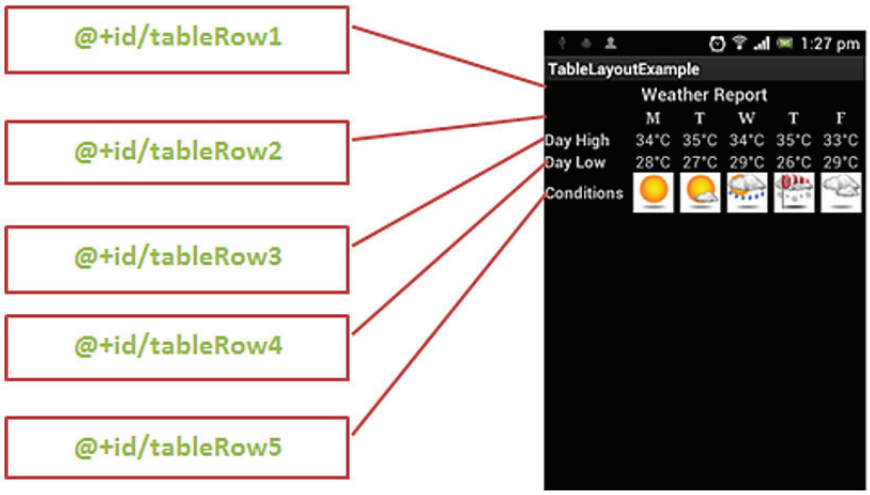


Figura 57

Para testar este layout, adicione outro arquivo de layout, conforme visto anteriormente, e dê o nome de layout_table. Substitua o código gerado pelo abaixo descrito:



Figura 58

```
<?xml version=”1.0” encoding=”utf-8”?>
<TableLayout xmlns:android=”http://schemas.android.com/apk/
res/android”
    android:layout_width=”match_parent”
    android:layout_height=”match_parent”
    android:stretchColumns=”1”>
    <TableRow>
        <TextView
            android:text=”Linha 1 - Coluna 1”
            android:padding=”3dip” />
        <TextView
            android:text=”Linha 1 - Coluna 2”
            android:gravity=”right”
            android:padding=”3dip” />
    </TableRow>

    <TableRow>
        <TextView
            android:text=”Linha 2 - Coluna 1”
            android:padding=”3dip” />
        <TextView
            android:text=”Linha 2 - Coluna 2”
            android:gravity=”right”
            android:padding=”3dip” />
    </TableRow>
</TableLayout>
```

Unidades que podem ser utilizadas para as dimensões:

Tipo	Descrição
px (pixels)	Correspondente ao número de pixels da tela
in (polegadas)	Baseado no tamanho físico da tela
mm (milímetros)	Baseado no tamanho físico da tela
pt(pontos)	1/72 de uma polegada, baseado no tamanho físico da tela
dpoudip	(Density-independent Pixels) Essa unidade é relativa à resolução da tela. Por exemplo se a resolução da tela é de 160 dpi, significa que um dp representa 1 pixel em um total de 160.
sp	(Scale-independent Pixels) Igual ao dp, mas também considera o tamanho da fonte que o usuário está utilizando. É recomendado que use essa unidade quando especificar o tamanho de uma fonte, para que esta seja automaticamente ajustada conforme as preferências da tela do usuário.

Figura 59

RelativeLayout

Como o próprio nome já diz, o Relative Layout é relativo. Mas o que isso quer dizer? Ao contrário do Linear Layout, que especifica sempre uma direção horizontal ou vertical, no relative layout, posicionamos os elementos por referência a outros elementos. Por exemplo, dizemos se o botão estará abaixo de um campo de texto, do lado direito ou até mesmo acima dele.

Neste tipo de layout colocamos os itens em posições relativas a outros itens. As tags android:layout_toRightOf, android:layout_toLeftOf, android:layout_below, android:layout_above permitem decidir a colocação relativa dos componentes.

Para testar este layout, adicione outro arquivo de layout, conforme visto anteriormente e dê o nome de layout_relative. Substitua o código gerado pelo abaixo descrito:



Figura 60

```
<?xml version=”1.0” encoding=”utf-8”?>
<RelativeLayout xmlns:android=”http://schemas.android.com/
apk/res/android”
    android:layout_width=”fill_parent”
    android:layout_height=”fill_parent”>
    <TextView
        android:id=”@+id/label”
        android:layout_width=”fill_parent”
        android:layout_height=”wrap_content”
        android:text=”Type here:”/>
    <EditText
        android:id=”@+id/entry”
        android:layout_width=”fill_parent”
        android:layout_height=”wrap_content”
        android:background=”@android:drawable/editbox_back-
ground”
        android:layout_below=”@id/label”/>
    <Button
        android:id=”@+id/ok”
        android:layout_width=”wrap_content”
        android:layout_height=”wrap_content”
        android:layout_below=”@id/entry”
        android:layout_alignParentRight=”true”
        android:layout_marginLeft=”10dip”
        android:text=”OK” />
    <Button
        android:layout_width=”wrap_content”
```

```
            android:layout_height="wrap_content"
            android:layout_toLeftOf="@id/ok"
            android:layout_alignTop="@id/ok"
            android:text="Cancel" />
    </RelativeLayout>
```

Observe, também, as tags para margem como **android:layout_marginLeft** e **android:layout_marginTop**



Veja mais detalhes em: http://developer.android.com/guide/topics/ui/layout/relative.html

Outras tags android: dentro da descrição XML

Podemos, também, configurar melhor nossos componentes, por meio de outros parâmetros: **android:textSize="18dip"**, **android:width="150px"**, **android:height="60px" android:textStyle="bold"**. Observe as tags que definem cores (`android:background` e `android:textColor`).

Para este exemplo, iremos dividir os layouts em 2 arquivos. Primeiramente, criaremos um arquivo de layout abaixo com o nome: listview_item.

```
    <?xml version="1.0" encoding="utf-8"?>
    <RelativeLayout xmlns:android="http://schemas.android.com/
    apk/res/android"
        android:layout_width="fill_parent"
        android:layout_height="50dp"
        android:background="@mipmap/ic_launcher" >

        <LinearLayout
            android:layout_width="fill_parent"
            android:layout_height="fill_parent"
            android:layout_alignParentBottom="true"
            android:orientation="vertical"
            android:padding="5dip" >
            <TextView
                android:id="@+id/title"
                android:layout_width="fill_parent"
                android:layout_height="wrap_content"
                android:singleLine="true"
                android:textSize="28sp"
                android:textStyle="bold"
                android:textColor="#0000ff"/>

        </LinearLayout>
        <TextView
            android:id="@+id/tvMsg"
            android:layout_width="wrap_content"
```

```
            android:layout_height="wrap_content"
            android:layout_alignParentRight="true"
            android:layout_centerVertical="true"
            android:layout_marginRight="10dp"
            android:textColor="#ebebeb"
            android:textSize="20sp"
            android:textStyle="bold" />

    </RelativeLayout>
```

Criaremos o segundo arquivo de layout com nome de layout_linear2 e iremos referenciar dentro dele o arquivo criado acima, conforme exemplo abaixo:

```
    <LinearLayout xmlns:android="http://schemas.android.com/
    apk/res/android"
        xmlns:tools="http://schemas.android.com/tools"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:background="#ebebeb"
        android:orientation="vertical" >
        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center_horizontal"
            android:layout_marginBottom="15dp"
            android:layout_marginLeft="15dp"
            android:layout_marginRight="15dp"
            android:layout_marginTop="15dp"
            android:background="#990000"
            android:orientation="vertical" >

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="5dp"
                android:background="#990000"
                android:orientation="vertical" >

                <TextView
                    android:id="@+id/textView"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginLeft="10dp"
                    android:text="Menu"
                    android:textAppearance="?android:attr/
    textAppearanceLarge"
                    android:textColor="#00ff00"
                    android:textSize="25sp"
                    android:textStyle="bold" />
                <Button
```

```
                android:id="@+id/button"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textSize="18dip"
                android:width="150px"
                android:height="60px"
                android:textStyle="bold"
                android:text="Botao" />

        </LinearLayout>

        <ListView
            android:id="@+id/list"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:layout_marginBottom="10dp"
            tools:listitem="@layout/listview_item" >

        </ListView>
    </LinearLayout>
</LinearLayout
```

Referência ao arquivo gerado acima

Perceba que podemos ter vários tipos de layouts, inclusive um dentro do outro. Também podemos criar arquivos separados de layout e realizar a referência dentro de outro arquivo de layout. O resultado será o layout abaixo:

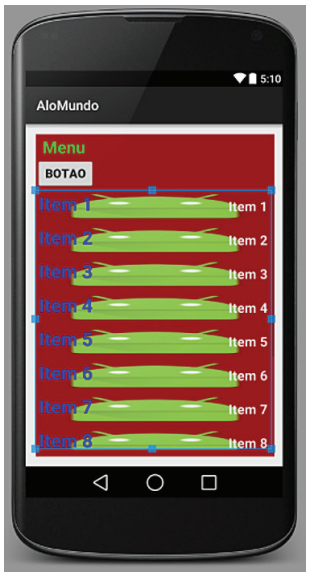


Figura 61

Caso você queira organizar melhor as cores, podemos criar um arquivo xml de cores e fazer a referência dentro dos arquivos acima criados. Para isso, crie um arquivo xml com o nome "mycolor" e o mesmo ficará disponível na pasta values, conforme abaixo e configure suas cores:

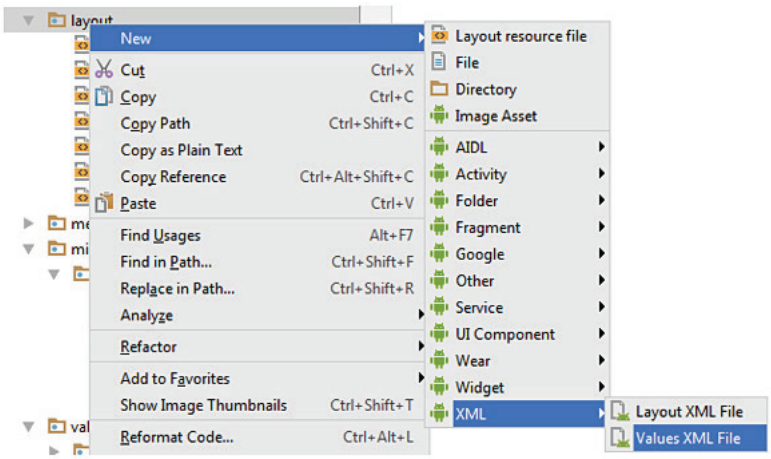


Figura 62

Nele, iremos acrescentar as configurações abaixo:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="red">#ff0000</color>
    <color name="green">#00ff00</color>
    <color name="blue">#0000ff</color>
</resources>
```

Agora, já podemos substituir o código hexadecimal da cor pela nomenclatura criada acima, conforme exemplo abaixo:

Substituir `android:textColor="#0000ff"` por `android:textColor="@color/blue"`

Como testar os layouts dos exemplos anteriores?

Para executar utilizando o novo layout, temos que mudar na programação a seleção desse novo layout, conforme abaixo:

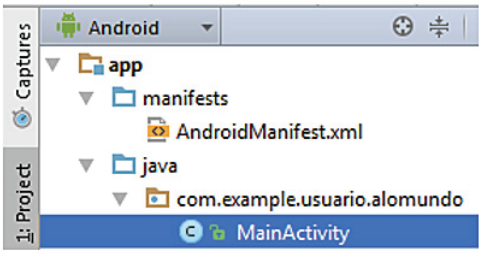


Figura 63

No programa MainActivity.java, altere no método onCreate a chamada ao novo layout, conforme abaixo:

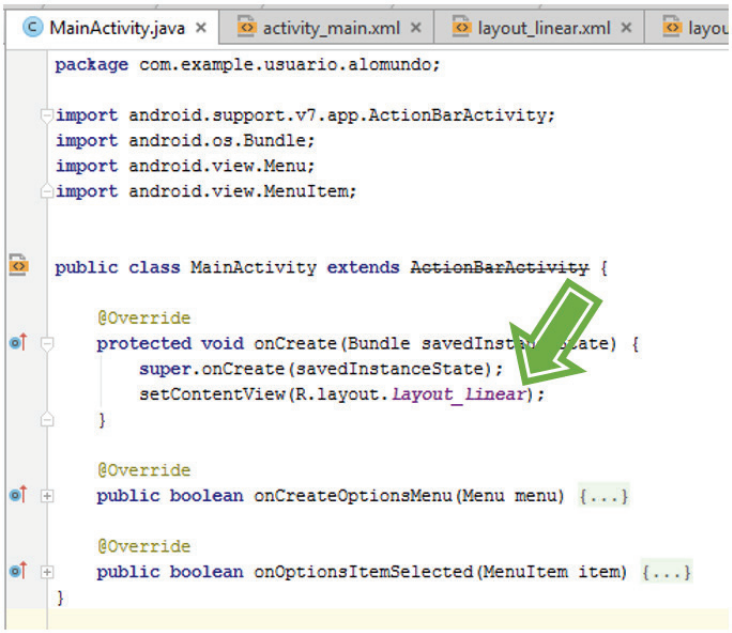


Figura 64

Agora, execute o projeto novamente e veja como ficará na tela do emulador do dispositivo.

Caso tenha fechado o emulador, ao executar, abrirá a tela abaixo, mas como já configuramos o emulador anteriormente, é só selecionar o Android virtual device e o botão OK.

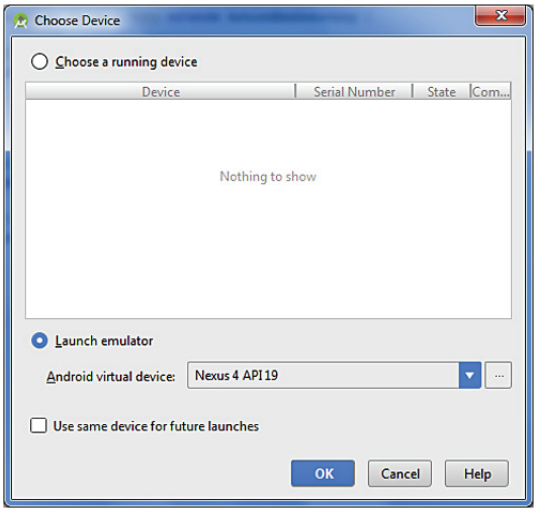


Figura 65

Ao executar, o resultado será o exibido abaixo:



Figura 66

Lembrando que temos criado todos os layouts abaixo e, caso queira, podemos alterar para qualquer um desses criados e executar o projeto novamente.

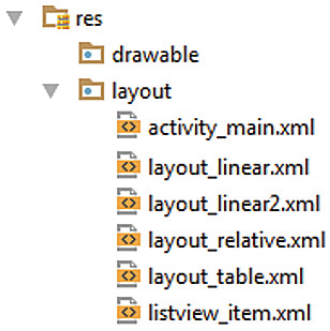


Figura 67

As tags dos arquivos .xml do Android

Apesar de que um conhecimento básico de XML facilitará entender melhor a sintaxe utilizada nos arquivos .xml que descrevem o visual de uma tela do programa Android, o ambiente ou IDE Android Studio ajudará nesta elaboração. Na figura, mostramos como o IDE Android Studio nos orienta na elaboração de um arquivo .xml. Para o componente de interface TextView mostrado, aparece uma lista de tags que definem as características deste objeto.

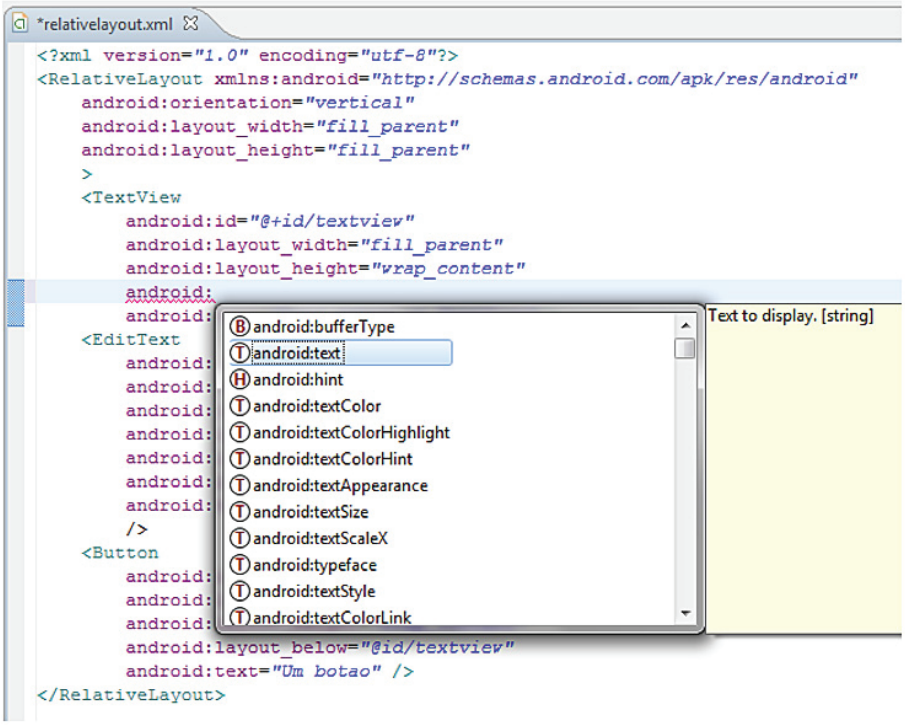


Figura 68

Outra possibilidade será consultar a ajuda detalhada de uma classe de componente de interface (novamente utilizamos TextView como exemplo). Observe, na parte inferior da figura, a lista de atributos XML, seus nomes e métodos relacionados.

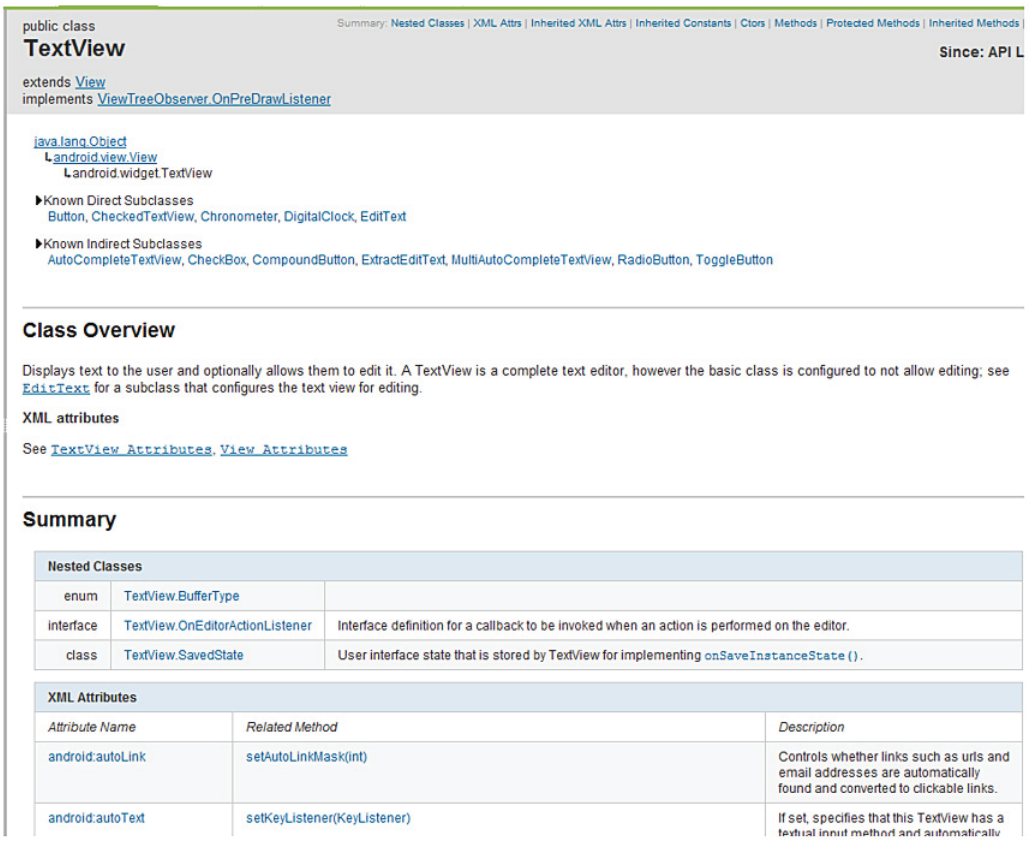


Figura 69

Vale lembrar que podemos gerar diversos tipos de layouts e vai da criatividade e necessidade de cada um para podermos elaborar um layout mais aprimorado. Abaixo, alguns exemplos de layouts mais sofisticados:

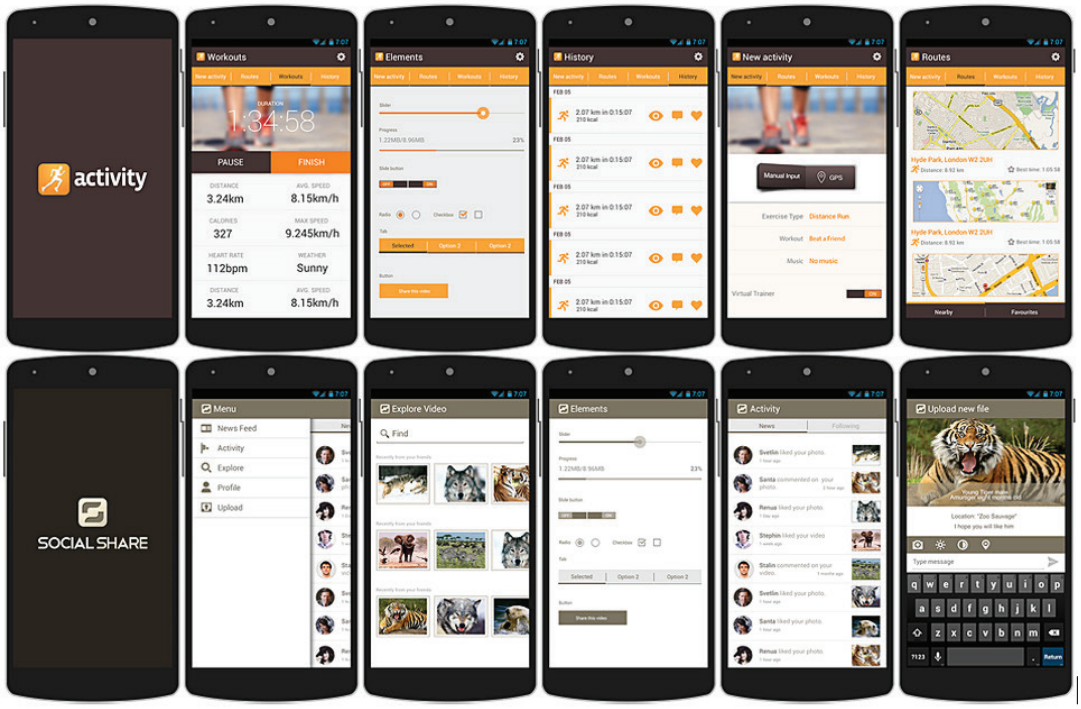


Figura 70

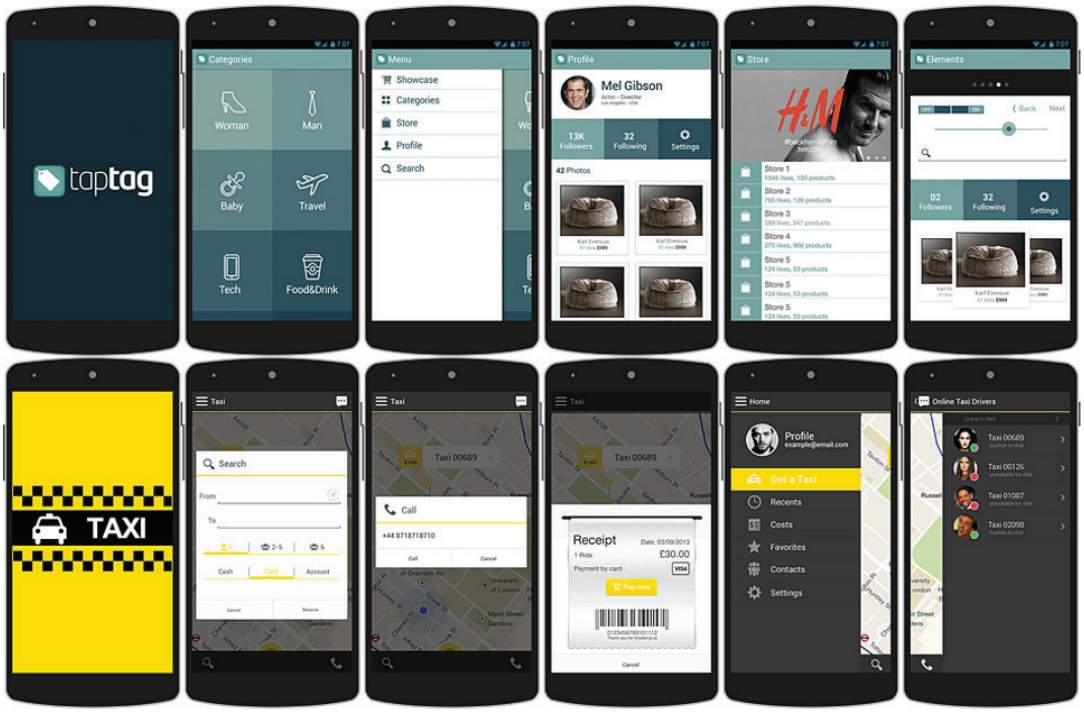


Figura 71


Select City
mapit
Please select a city
Sydney
Melbourne
Home
Get Me Somewhere
My Home
My Work Places
Starbucks coffee
John's house
Route Details
Walk
Bray Street
Pulteney Street
Pitt Street
Results
Get Route
King William Road
Chatt
Sign in with
Email
Password
Register
Log In
Project
My Special project
iOS App developer
Buy
Vacancy
Friday Meeting
Too many expenses
Last Party
Note
Very important!
Steps
To hurry up
Evaluate with wife
Study the case
Apartment block
Birthday dinner
Chat
Group Chat

Figura 72

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Sites

Layouts
https://goo.gl/NnYeMS

Table
https://goo.gl/tBey8L

Compatibilidade com várias telas
https://goo.gl/oaxTnQ

Relative Layout
https://goo.gl/hnav25

Referências

ANDROID. **Android Developers**. Disponível em http://developer.android.com

LECHETA, R. R. **Google Android, Aprenda a criar aplicações para dispositivos móveis com o Android SDK**. São Paulo: Novatec, 2010.

MEDNIEKS, Z. et. al. **Desenvolvimento de Aplicações Android**. São Paulo: Novatec, 2009.

LEE, W. M. **Introdução ao Desenvolvimento de Aplicativos para o Android**. Rio de Janeiro: Ciência Moderna, 2011.

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000


Cruzeiro do Sul
Educacional